

Project Report

AttackDNA

Organization-Specific Behavioral Detection Engine

Name : Amber Waseem

Reference Code: ARZ-2026-UDEB

Prepared by: Amber
Submitted to : Wardet Wahaj

Dataset: CICIDS2017 (Canadian Institute for Cybersecurity, UNB)
GitHub Repository:
<https://github.com/Amberr8/attackdna>

Abstract

AttackDNA is an anomaly based network intrusion detection system that learns a behavioral baseline for each individual host on a network and flags deviations from that host's own normal pattern, rather than applying one generic rule set to every host. The system is built around per entity Isolation Forest models trained on the CICIDS2017 public dataset, with a full pipeline covering data cleaning, feature selection, model training, detection, statistical explanation generation, MITRE ATT&CK technique mapping, and an interactive Streamlit dashboard. Following an initial remediation phase that addressed twelve identified limitations, the project supervisor requested four further improvements: adding time window features to close a detection gap on port scanning and botnet traffic, benchmarking a second algorithm (Local Outlier Factor), validating results across multiple random seeds, and diagnosing the root cause of the baseline false positive rate. This report documents the complete system, all source code, and the results of both phases of work, including negative and mixed findings reported alongside positive ones.

Contents

1	Introduction	2
1.1	Problem Statement.....	2
1.2	Objectives.....	2
2	Dataset	2
2.1	CICIDS2017 Overview.....	2
2.2	Days Used.....	2
2.3	Data Quality Issues Identified and Fixed.....	3
3	System Design	3
3.1	Defining an Entity.....	3
3.2	Pipeline Overview.....	4
4	Implementation: Core Pipeline Source Code	6
4.1	data_prep.py: Data Cleaning and Entity Extraction.....	6
4.2	feature_selection.py: Feature Analysis.....	7
4.3	train.py: Per-Entity Model Training.....	9
4.4	detect.py: Scoring.....	12
4.5	evaluate.py: Standard Performance Metrics.....	13
4.6	explain.py: Statistical Explanation Generation.....	15
4.7	mitre_map.py: MITRE ATT&CK Technique Mapping (Final Version).....	17
4.8	dashboard/app.py: Interactive Streamlit Dashboard (Final Version).....	19
5	Initial Results, Before Supervisor Review	22
5.1	Detection Rate by True Label.....	22
5.2	The 192.168.10.50 Outlier.....	22
6	Limitations Identified and Addressed	22
6.1	Baseline Expansion (Item 1).....	23
6.2	Threshold Calibration Comparison (Item 3).....	23
6.3	Rule-Based MITRE Mapping Improvement (Item 6).....	23
6.4	Extended Evaluation (Item 9).....	24
7	Supervisor-Requested Improvements	25
7.1	Time Window Features (Highest Priority).....	25
7.1.1	Motivation.....	25
7.1.2	Design and Implementation.....	25
7.1.3	Verification Before Retraining.....	26
7.1.4	First Attempt: Retraining Isolation Forest Directly Did Not Work.....	27
7.1.5	Root Cause Diagnosis.....	27
7.1.6	Fix: A Dedicated Rule-Based Scan Detector.....	27
7.1.7	Scan Detector Results (Standalone).....	28
7.1.8	Combining the Scan Detector with Isolation Forest.....	29
7.1.9	Final Combined System Results.....	30

7.2	Local Outlier Factor Comparison	31
7.2.1	Motivation and Conceptual Difference	31
7.2.2	Implementation	31
7.2.3	Results: LOF versus Isolation Forest	34
7.3	Split and Seed Stability Testing	36
7.3.1	Motivation	36
7.3.2	Method	36
7.3.3	Results	37
7.3.4	Interpretation	37
7.4	Root-Cause Diagnosis of the False Positive Rate	39
7.4.1	Motivation	39
7.4.2	Method	39
7.4.3	Finding: The False Positive Rate Is Concentrated in Two Entities	39
7.4.4	Score Distribution Sanity Check	39
7.4.5	Entity 192.168.10.50: Attack-Adjacent Traffic Distortion	40
7.4.6	Entity 192.168.10.25: Reduced Activity Relative to Baseline	40
7.4.7	Conclusion	40
7.5	Dashboard Updates	40
7.6	Summary of Supervisor-Requested Improvements	41
8	Dashboard Screenshots	42
9	Discussion	46
10	Conclusion and Future Work	46
10.1	Summary	46
10.2	Recommended Priority for Future Work	46
10.3	Closing Note	47

1 Introduction

1.1 Problem Statement

Traditional intrusion detection systems rely on generic, static rules, for example Sigma, Snort, or YARA signatures, that apply the same definition of suspicious activity to every host in a network. This causes two well known problems in real Security Operations Centers. First, high false positive rates: a rule tuned for a quiet workstation will constantly misfire on a busy file server, and vice versa. Second, blind spots to novel attacks: signature based tools can only detect what they already have a rule for.

AttackDNA addresses this by learning a behavioral baseline per entity rather than applying one global rule set, so that normal is defined relative to each host's own historical behavior.

1.2 Objectives

Build a working, genuinely trained machine learning pipeline for anomaly detection, not an API wrapper.

Detect attacks without ever training on attack labeled data, using a fully unsupervised approach.

Produce human readable explanations for every flagged anomaly rather than a binary alert.

Map detected anomalies to standardized MITRE ATT&CK techniques.

Present results through an interactive analyst facing dashboard.

Operate entirely on CPU hardware with free, open source tools and no dependency on always on live infrastructure.

2 Dataset

2.1 CICIDS2017 Overview

CICIDS2017, published by the Canadian Institute for Cybersecurity at the University of New Brunswick, is a labeled network traffic dataset captured over five days on a real network testbed. Traffic was processed by CICFlowMeter into flow records, where each row summarizes one network conversation using approximately 79 numeric statistics such as duration, packet counts, byte rates, inter arrival timing, and TCP flag counts, together with a ground truth Label column.

Two versions of this dataset are published. The version initially downloaded for this project, named MachineLearningCSV, excludes IP address columns, which made per entity modeling impossible. This was identified early and corrected by switching to the GeneratedLabelled-Flows version, which retains Flow ID, Source IP, Source Port, Destination IP, Destination Port, Protocol, and Timestamp alongside the same flow statistics, for 85 columns in total.

2.2 Days Used

Day / File	Role	Content
Monday-WorkingHours	Baseline, training	100 percent benign traffic
Tuesday-WorkingHours	Baseline, BENIGN rows only	Brute force present, filtered to BENIGN

Day / File	Role	Content
Wednesday-workingHours	Held out test	DoS Hulk, GoldenEye, Slowloris, Slowhttptest, Heartbleed
Thursday-Morning-WebAttacks	Baseline, BENIGN rows only	Web attacks present, filtered to BENIGN
Thursday-Afternoon-Infiltration	Baseline, BENIGN rows only	Infiltration present, filtered to BENIGN
Friday-Morning	Held out test	Botnet traffic
Friday-Afternoon-PortScan	Held out test	Port scanning
Friday-Afternoon-DDoS	Held out test	DDoS traffic

The final training baseline combines Monday, which is fully benign, with the BENIGN labeled rows from Tuesday and both Thursday files, totaling 2,106,675 benign training rows across 15 internal entities. No attack labeled row, and no row from Wednesday or Friday, was included in training, which avoids data leakage between training and evaluation.

2.3 Data Quality Issues Identified and Fixed

Column headers contain leading whitespace, for example a column literally named ' Flow Duration'. Fixed by stripping all column names on load.

Flow Bytes per second and Flow Packets per second become infinite when Flow Duration is zero, since CICFlowMeter still writes a division result even when the divisor is zero. Fixed by converting infinite values to missing values and dropping the affected rows, a small fraction of the data.

Some Label values contain a non UTF-8 byte, hex 0x96, a corrupted en dash character. Fixed by normalizing the Label string after load.

The file Thursday-Morning-WebAttacks specifically fails to load under default UTF-8 decoding because of this byte. Fixed by reading the file with latin1 encoding, which maps every byte to a character without failing, after which the existing label normalization step repairs the character correctly.

Exact duplicate rows exist in the raw files. Removed using pandas drop_duplicates.

3 System Design

3.1 Defining an Entity

A core requirement of AttackDNA is per entity behavioral modeling, but CICIDS2017 does not label which IP address in a flow is the internal host being monitored. Investigation showed that CICFlowMeter assigns the Source and Destination columns based on which side sent the first packet of the flow, not based on internal versus external network membership. As a result, a single internal host's traffic is split across both the Source IP and Destination IP columns.

This was confirmed empirically on a 200,000 row sample of Monday's traffic.

Check	Count	Percent
Flows where Source IP is internal (192.168.10.x)	161,018	80.5%
Flows where Destination IP is internal	89,380	44.7%

Check	Count	Percent
Flows where both sides are internal	50,522	25.3%
Flows where neither side is internal (edge case)	124	0.06%

Design decision: an entity column was derived as whichever IP address in the flow belongs to the internal subnet 192.168.10.0/24, regardless of which raw column it appears in. When both sides of a flow are internal, the flow is counted once for each participating entity, since it is genuine traffic for both hosts. Flows where neither side is internal, 0.06 percent of the data and likely a capture artifact, are dropped. This design decision was essential, since without it roughly half of every entity's real traffic would have been silently excluded from its behavioral profile.

3.2 Pipeline Overview

The system is implemented as a sequence of independent Python scripts, each reading the previous stage's output and writing its own output file. Any stage can be re-run, inspected, or debugged in isolation, and intermediate results remain available on disk for verification.

Core pipeline (initial build):

`data_prep.py`, loads a raw CICIDS2017 CSV, cleans it, and derives the entity column.

`feature_selection.py`, analyzes the benign baseline to identify zero variance and highly redundant features.

`train.py`, trains one Isolation Forest model per entity on benign baseline data, with a calibrated per entity anomaly threshold.

`detect.py`, loads each entity's saved model and scores new held out traffic.

`evaluate.py`, computes precision, recall, F1 score, and false positive rate from scored output.

`explain.py`, generates a plain language, statistical explanation for every flagged anomaly.

`mitre_map.py`, maps each explanation to a MITRE ATT&CK technique using a rule based evidence scoring system.

`dashboard/app.py`, a Streamlit application presenting all results interactively.

Extended pipeline (supervisor requested improvements, Section 7):

`time_window_features.py`, adds connection burst features per entity per minute.

`diagnose_entity.py`, compares a single entity's baseline versus attack period statistics.

`scan_detector.py`, a dedicated rule based detector for burst and scan patterns.

`combine_detectors.py`, merges Isolation Forest predictions with the scan detector into one final flag.

`diagnose_fpr.py`, breaks down the false positive rate by entity and by anomaly score distribution.

`train_lof.py` and `detect_lof.py`, an alternative anomaly detection algorithm for comparison.

`seed_stability.py`, repeats training and evaluation across multiple random seeds to test result stability.

All code presented in this report is the final, working version as executed on the project's Ubuntu 24.04 LTS environment, using a Python virtual environment, CPU only, 16 gigabytes of RAM.

4 Implementation: Core Pipeline Source Code

4.1 data_prep.py: Data Cleaning and Entity Extraction

This is the final version of the cleaning script, including the latin1 encoding fix and the row_id column added later to support reliable joins between detectors (Section 7.1).

```
import pandas as pd
import numpy as np
import sys

def load_raw_csv(path):
    """Load the CSV and strip whitespace from column names.
    CICIDS2017 CSVs have leading spaces in most headers, e.g.
    * Flow Duration' instead of 'Flow Duration'. Some files (notably
    Thursday-Morning-WebAttacks) contain a non-UTF-8 byte (0x96) in
    the Label column, so we read as latin1 - this maps every byte
    1-to-1 to a character instead of failing, and keeps \x96 as the
    literal character normalize_labels() already knows how to fix."""
    df = pd.read_csv(path, low_memory=False, encoding="latin1")
    df.columns = df.columns.str.strip()
    return df

def extract_entity(df):
    """Figure out which IP in each flow is the internal host, since
    CICFlowMeter assigns Source/Destination based on who sent the first
    packet - not based on internal vs external. So a given internal
    host's traffic is split across both columns. We fix that by adding
    an 'entity' column that always points to the internal host.

    If both sides are internal (two internal hosts talking), the flow
    is counted for both entities, since it's real traffic for both."""
    internal_prefix = "192.168.10."

    src_internal = df["Source IP"].str.startswith(internal_prefix)
    dst_internal = df["Destination IP"].str.startswith(internal_prefix)

    src_rows = df[src_internal].copy()
    src_rows["entity"] = src_rows["Source IP"]

    dst_rows = df[dst_internal].copy()
    dst_rows["entity"] = dst_rows["Destination IP"]

    combined = pd.concat([src_rows, dst_rows], ignore_index=True)

    dropped = len(df) - (src_internal | dst_internal).sum()
    print(f"Dropped {dropped} flows with neither side internal.")

    return combined

def clean_flow_stats(df):
    """Fix inf and NaN values in the numeric columns. Flow Bytes/s and
    Flow Packets/s can become 'inf' when Flow Duration is 0, since you
    can't divide by zero - CICFlowMeter still writes 'inf' into the file
    when that happens."""
    numeric_cols = df.select_dtypes(include=[np.number]).columns
    df[numeric_cols] = df[numeric_cols].replace([np.inf, -np.inf], np.nan)
    before = len(df)
    df = df.dropna(subset=numeric_cols)
    print(f"Dropped {before - len(df)} rows with NaN/inf values.")
```

```

return df

def normalize_labels(df):
    """Clean up the Label column - remove extra spaces and fix a known
    broken character that shows up in some attack labels."""
    df["Label"] = df["Label"].astype(str).str.strip().str.replace("\x96", "-", regex=False)
    return df

def drop_duplicates(df):
    before = len(df)
    df = df.drop_duplicates()
    print(f"Dropped {before - len(df)} duplicate rows.")
    return df

if __name__ == "__main__":
    input_path = sys.argv[1]
    output_path = sys.argv[2]
    df = load_raw_csv(input_path)
    print(f"Loaded {len(df)} rows, {len(df.columns)} columns.")
    df = normalize_labels(df)
    df = drop_duplicates(df)
    df = extract_entity(df)
    df = clean_flow_stats(df)
    df["row_id"] = range(len(df))
    print(f"Final cleaned dataset: {len(df)} rows.")
    print(f"Unique entities: {df['entity'].nunique()}")
    print(df["Label"].value_counts())
    df.to_csv(output_path, index=False)
    print(f"Saved to {output_path}")

```

4.2 feature_selection.py: Feature Analysis

Built to identify redundant and uninformative features. Identifies zero variance features, constant across all traffic and carrying no information, and pairs of features with correlation above 0.95, redundant since one can be derived from the other.

```

import pandas as pd
import numpy as np
import sys

df = pd.read_csv(sys.argv[1])
df = df[df["Label"] == "BENIGN"]
non_features = ["Flow ID", "Source IP", "Destination IP", "Timestamp",
                "Source Port", "Label", "entity"]
feature_cols = [c for c in df.columns if c not in non_features]

X = df[feature_cols].select_dtypes(include=[np.number])
variances = X.var().sort_values()
print("--- Lowest-variance features (candidates to drop) ---")
print(variances.head(10))

corr = X.corr().abs()
upper = corr.where(np.triu(np.ones(corr.shape), k=1).astype(bool))
high_corr = [(col, row, upper.loc[row, col]) for col in upper.columns
              for row in upper.index if upper.loc[row, col] > 0.95]
print(f"\n--- {len(high_corr)} feature pairs with correlation > 0.95 (redundant) ---")
for a, b, v in high_corr[:15]:
    print(f"{a} <-> {b}: {v:.3f}")

```

Result: 22 features were identified for removal, 10 zero variance features such as CWE Flag Count and Bwd URG Flags, and 12 redundant correlated features such as Total Backward Packets and Max Packet Length. This reduced the feature count from 79 to 57 with negligible impact on F1 score, moving network wide F1 from 0.743 to 0.742.

4.3 train.py: Per-Entity Model Training

Incorporates the multi day baseline, the reduced feature set, and robustness handling for small or edge case entities.

```
import sys
import pandas as pd
import numpy as np
import pickle
import os
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler

NON_FEATURE_COLUMNS = [
    "Flow ID", "Source IP", "Destination IP", "Timestamp",
    "Source Port", "Label", "entity",
    # Zero variance - constant across all traffic, no information
    "CWE Flag Count", "Fwd Avg Bulk Rate", "Bwd Avg Packets/Bulk",
    "Bwd Avg Bulk Rate", "Bwd Avg Bytes/Bulk", "Bwd URG Flags",
    "Fwd Avg Packets/Bulk", "Fwd Avg Bytes/Bulk", "Fwd URG Flags",
    "Bwd PSH Flags",
    # Redundant - correlation > 0.95 with a kept feature
    "Total Backward Packets", "Total Length of Bwd Packets",
    "Fwd IAT Total", "Fwd IAT Max", "Fwd IAT Min",
    "Bwd IAT Total", "Bwd IAT Mean", "Fwd Packets/s",
    "Max Packet Length", "SYN Flag Count", "ECE Flag Count",
    "Average Packet Size"
]

def get_feature_columns(df):
    """Everything except IDs, timestamps, ports, and the label is a feature."""
    return [col for col in df.columns if col not in NON_FEATURE_COLUMNS]

def train_entity_model(entity_df, feature_cols, min_flows=10, target_fpr=0.05):
    if len(entity_df) < min_flows:
        print(f" Skipping - only {len(entity_df)} flows (need {min_flows}+).")
        return None, None, None

    # Handle missing or infinite values
    X_df = entity_df[feature_cols].select_dtypes(include=[np.number])
    X_df = X_df.replace([np.inf, -np.inf], np.nan).fillna(0)

    shuffled = X_df.sample(frac=1, random_state=42)
    split_point = int(len(shuffled) * 0.8)

    # Fallback to full set if train or validation split is too small
    if split_point == 0 or split_point == len(shuffled):
        train_df = shuffled
        val_df = shuffled
    else:
        train_df = shuffled.iloc[:split_point]
        val_df = shuffled.iloc[split_point:]

    X_train = train_df.values
    X_val = val_df.values

    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_val_scaled = scaler.transform(X_val)

    model = IsolationForest(
        n_estimators=100,
```

```

        contamination="auto",
        random_state=42,
        n_jobs=-1
    )
    model.fit(X_train_scaled)

    val_scores = model.decision_function(X_val_scaled)
    threshold = np.percentile(val_scores, target_fpr * 100)

    return model, scaler, threshold

def save_entity_model(entity_ip, model, scaler, threshold, feature_cols, out_dir="models"):
    os.makedirs(out_dir, exist_ok=True)
    safe_name = entity_ip.replace(".", "_")
    path = os.path.join(out_dir, f"{safe_name}.pkl")
    bundle = {
        "model": model,
        "scaler": scaler,
        "threshold": threshold,
        "feature_cols": feature_cols,
        "entity_ip": entity_ip
    }
    with open(path, "wb") as f:
        pickle.dump(bundle, f)
    print(f" Saved -> {path} (threshold={threshold:.4f})")

if __name__ == "__main__":
    input_paths = sys.argv[1:] if len(sys.argv) > 1 else ["data/processed/monday_clean.csv"]
    print(f>Loading baseline from {len(input_paths)} file(s)...")

    frames = []
    for path in input_paths:
        if not os.path.exists(path):
            print(f" Warning: File not found: {path}")
            continue
        day_df = pd.read_csv(path)
        day_df.columns = day_df.columns.str.strip()
        before = len(day_df)
        day_df = day_df[day_df["Label"] == "BENIGN"]
        print(f" {path}: {before} rows -> {len(day_df)} BENIGN rows")
        frames.append(day_df)

    if not frames:
        print("No valid baseline data found!")
        sys.exit(1)

    df = pd.concat(frames, ignore_index=True)
    print(f"Combined baseline: {len(df)} BENIGN rows, {df['entity'].nunique()} entities\n")

    feature_cols = get_feature_columns(df)
    print(f"Using {len(feature_cols)} feature columns.")

    entities = df["entity"].unique()
    print(f"Training models for {len(entities)} entities...\n")

    trained_count = 0
    skipped_count = 0

    for entity_ip in entities:
        print(f"Entity: {entity_ip}")
        entity_df = df[df["entity"] == entity_ip]
        model, scaler, threshold = train_entity_model(entity_df, feature_cols)
        if model is None:
            skipped_count += 1
            continue

```

```
save_entity_model(entity_ip, model, scaler, threshold, feature_cols)
trained_count += 1

print(f"\nDone. Trained: {trained_count}, Skipped: {skipped_count}")
```

4.4 detect.py: Scoring

```
import pandas as pd
import numpy as np
import pickle
import os
import sys

def load_entity_model(entity_ip, models_dir="models"):
    """Load a saved model+scaler+threshold bundle for one entity. Returns
    None if no model exists for this entity."""
    safe_name = entity_ip.replace(".", "_")
    path = os.path.join(models_dir, f"{safe_name}.pkl")

    if not os.path.exists(path):
        return None

    with open(path, "rb") as f:
        bundle = pickle.load(f)

    return bundle

def score_entity(entity_df, bundle):
    """Score all of one entity's flows using its trained model and its
    own calibrated threshold (not Isolation Forest's built-in default).
    A flow is flagged as anomalous if its score falls below the
    entity's threshold, which was calibrated to target a 5% false
    positive rate on clean validation data."""
    model = bundle["model"]
    scaler = bundle["scaler"]
    threshold = bundle["threshold"]
    feature_cols = bundle["feature_cols"]

    X = entity_df[feature_cols].values
    X_scaled = scaler.transform(X)

    anomaly_score = model.decision_function(X_scaled)
    predicted = np.where(anomaly_score < threshold, -1, 1)

    entity_df = entity_df.copy()
    entity_df["predicted"] = predicted
    entity_df["anomaly_score"] = anomaly_score

    return entity_df

if __name__ == "__main__":
    input_path = sys.argv[1]

    print(f"Loading {input_path} ...")
    df = pd.read_csv(input_path)
    print(f"Loaded {len(df)} rows.")

    entities = df["entity"].unique()
    print(f"Scoring {len(entities)} entities...\n")

    results = []
    no_model_count = 0

    for entity_ip in entities:
        bundle = load_entity_model(entity_ip)
```

```

    if bundle is None:
        print(f"Entity {entity_ip}: no model found, skipping.")
        no_model_count += len(df[df["entity"] == entity_ip])
        continue

    entity_df = df[df["entity"] == entity_ip]
    scored_df = score_entity(entity_df, bundle)
    results.append(scored_df)

    n_anomalies = (scored_df["predicted"] == -1).sum()
    print(f"Entity {entity_ip}: {len(scored_df)} flows, "
          f"{n_anomalies} flagged as anomalies.")

all_results = pd.concat(results, ignore_index=True)

print(f"\nRows skipped (no model): {no_model_count}")
print(f"Total scored: {len(all_results)}")

print("\n--- Detection results by true label ---")
summary =
    all_results.groupby("Label")["predicted"].apply( lambda x:
        (x == -1).sum()
    )
total_by_label = all_results["Label"].value_counts()

for label in total_by_label.index:
    flagged = summary.get(label, 0)
    total = total_by_label[label]
    print(f"{label}: {flagged}/{total} flagged as anomaly "
          f"({flagged/total:.1%}")

output_path = sys.argv[2] if len(sys.argv) > 2 else "data/processed/wednesday_scored.csv"
all_results.to_csv(output_path, index=False)

```

4.4 evaluate.py: Standard Performance Metrics

Computes true positive, false positive, false negative, and true negative counts, and derived precision, recall, F1 score, and false positive rate, both network wide and per entity.

```

import pandas as pd
import sys

def compute_metrics(df, label_col="Label", pred_col="predicted"):
    """Compute precision, recall, F1, and FPR treating any non-BENIGN
    label as the positive (attack) class. df must already contain the
    true label and the model's prediction (-1 = anomaly, 1 = normal)."""
    is_attack = df[label_col] != "BENIGN"
    is_flagged = df[pred_col] == -1

    tp = (is_attack & is_flagged).sum()
    fn = (is_attack & ~is_flagged).sum()
    fp = (~is_attack & is_flagged).sum()
    tn = (~is_attack & ~is_flagged).sum()

    precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
    recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
    f1 = 2 * precision * recall / (precision + recall) if (precision + recall) > 0 else 0.0
    fpr = fp / (fp + tn) if (fp + tn) > 0 else 0.0

    return {

```



```

        "TP": tp, "FP": fp, "FN": fn, "TN": tn,
        "Precision": precision, "Recall": recall,
        "F1": f1, "FPR": fpr
    }

def print_metrics(name, metrics):
    print(f"\n--- {name} ---")
    print(f"TP={metrics['TP']} FP={metrics['FP']} "
          f"FN={metrics['FN']} TN={metrics['TN']}")
    print(f"Precision: {metrics['Precision']:.3f}")
    print(f"Recall: {metrics['Recall']:.3f}")
    print(f"F1-score: {metrics['F1']:.3f}")
    print(f"FPR: {metrics['FPR']:.3f}")

if __name__ == "__main__":
    scored_path = sys.argv[1]
    df = pd.read_csv(scored_path)

    print(f"Loaded {len(df)} scored rows from {scored_path}")

    overall = compute_metrics(df)
    print_metrics("Network-wide", overall)

    target_entity = "192.168.10.50"
    entity_df = df[df["entity"] == target_entity]
    if len(entity_df) > 0:
        entity_metrics = compute_metrics(entity_df)
        print_metrics(f"Entity {target_entity}", entity_metrics)
    else:
        print(f"\nNo rows found for entity {target_entity}")

```

4.5 explain.py: Statistical Explanation Generation

For every anomaly flagged during detection, this script compares the flow's feature values against that entity's baseline mean and standard deviation, computing a z-score per feature. The top deviating features are reported in plain language, for example Flow Duration is 8.2 times higher than normal, z equals 6.1. Computation is fully vectorized using per entity NumPy array batches rather than looped row by row, which reduced runtime on roughly 204,000 anomalies from over an hour, in an initial row by row version that was manually interrupted, to under one minute.

```
import pandas as pd
import numpy as np
import sys

NON_FEATURE_COLUMNS = [
    "Flow ID", "Source IP", "Destination IP", "Timestamp",
    "Source Port", "Label", "entity", "predicted", "anomaly_score"
]

def get_feature_columns(df):
    return [col for col in df.columns if col not in NON_FEATURE_COLUMNS]

def build_baseline_stats(monday_df, feature_cols):
    """For each entity, compute mean and std of every feature."""
    means = monday_df.groupby("entity")[feature_cols].mean()
    stds = monday_df.groupby("entity")[feature_cols].std()
    return means, stds

def explain_entity_batch(entity_df, feature_cols, mean_row, std_row, top_n=3):
    """Vectorized explanation for ALL anomalies of one entity at once,
    instead of looping row by row."""
    X = entity_df[feature_cols].values.astype(float)
    mean_arr = mean_row[feature_cols].values.astype(float)
    std_arr = std_row[feature_cols].values.astype(float)

    std_arr_safe = np.where((std_arr == 0) | np.isnan(std_arr), np.nan, std_arr)

    z_scores = np.abs(X - mean_arr) / std_arr_safe
    z_scores = np.nan_to_num(z_scores, nan=0.0)

    top_idx = np.argsort(-z_scores, axis=1)[: , :top_n]

    explanations = []
    for row_i in range(len(entity_df)):
        parts = []
        for col_i in top_idx[row_i]:
            z = z_scores[row_i, col_i]
            if z < 1:
                continue
            col_name = feature_cols[col_i]
            actual = X[row_i, col_i]
            mean = mean_arr[col_i]
            direction = "higher" if actual > mean else "lower"
            if mean != 0:
                ratio = actual / mean
                parts.append(f"{col_name} is {abs(ratio):.1f}x {direction} than normal (z={z:.1f})")
            else:
                parts.append(f"{col_name} is {actual} than normal (z={z:.1f})")
        explanations.append(" ".join(parts))
```

```

        parts.append(f"{col_name} is unusually {direction} (z={z:.1f})")
    if not parts:
        explanations.append("Deviation detected but no single feature stands out strongly.
")
    else:
        explanations.append("; ".join(parts))

return explanations

def compute_risk_score(anomaly_score, entity_criticality, mitre_severity, threat_intel=0):
    """anomaly_score: normalized 0-100 (already have this).
    entity_criticality: 0-100, assigned manually (e.g. web server=90).
    mitre_severity: 0-100 from the MITRE mapping step.
    threat_intel: 0-100, optional external feed score, default 0."""
    return (0.40 * anomaly_score +
            0.30 * entity_criticality +
            0.20 * mitre_severity +
            0.10 * threat_intel)

if __name__ == "__main__":
    baseline_paths = sys.argv[1].split(",")
    scored_path = sys.argv[2]
    output_path = sys.argv[3]
    print(f>Loading baseline from {len(baseline_paths)} file(s)...)
    monday_df = pd.concat([pd.read_csv(p) for p in baseline_paths], ignore_index=True)
    monday_df["Label"] = "BENIGN"

    print(f>Loading scored data: {scored_path} ...)
    scored_df = pd.read_csv(scored_path)

    feature_cols = get_feature_columns(monday_df)
    print(f>Using {len(feature_cols)} feature columns.")

    print("Building per-entity baseline statistics...")
    means, stds = build_baseline_stats(monday_df, feature_cols)

    anomalies = scored_df[scored_df["predicted"] == -1].copy()
    print(f>Explaining {len(anomalies)} flagged anomalies (vectorized)...)

    all_explanations = pd.Series(index=anomalies.index, dtype=object)

    for entity_ip, entity_df in anomalies.groupby("entity"):
        if entity_ip not in means.index:
            all_explanations.loc[entity_df.index] = "No baseline available for this entity."
            continue
        mean_row = means.loc[entity_ip]
        std_row = stds.loc[entity_ip]
        explanations = explain_entity_batch(entity_df, feature_cols, mean_row, std_row)
        all_explanations.loc[entity_df.index] = explanations
        print(f> Entity {entity_ip}: explained {len(entity_df)} anomalies.")

    anomalies["explanation"] = all_explanations
    anomalies.to_csv(output_path, index=False)
    print(f>\nSaved {len(anomalies)} explained anomalies to {output_path}")

```

4.6 mitre_map.py: MITRE ATT&CK Technique Mapping (Final Version)

A rule based, not learned, classifier that maps each anomaly's explanation to a MITRE ATT&CK technique. This final version includes the scan feature category added in Section 7.5 to recognize port scanning and burst patterns, alongside the original timing and volume categories. All features mentioned in an explanation are parsed and scored, not only the single top feature, and the category with the strongest combined evidence wins.

```
import pandas as pd
import re
import sys

TIMING_FEATURES = [
    "Fwd IAT Std", "Bwd IAT Std", "Fwd IAT Min", "Bwd IAT Min",
    "Idle Min", "Idle Std", "Active Min", "Active Std"
]

VOLUME_FEATURES = [
    "Bwd Packet Length Std", "Bwd Packet Length Mean",
    "Fwd Packet Length Std", "Fwd Packet Length Max",
    "Packet Length Variance", "Packet Length Mean",
    "FIN Flag Count"
]

SCAN_FEATURES = ["connections_per_minute", "distinct_ports_per_minute"]

def parse_explanation(explanation):
    """Break 'FeatureA is 3.2x higher than normal (z=4.1); FeatureB is ...'
    into a list of (feature_name, z_score) pairs."""
    if not isinstance(explanation, str):
        return []
    parsed = []
    for part in explanation.split("; "):
        if " is " not in part:
            continue
        feature = part.split(" is ")[0].strip()
        match = re.search(r"z=(\d.+)", part)
        z = float(match.group(1)) if match else 0.0
        parsed.append((feature, z))
    return parsed

def map_to_mitre(explanation):
    """Score ALL features mentioned in the explanation by category
    (scan, timing, volume), and classify based on which category has
    the strongest combined evidence. The scan check runs first, since
    a burst/scan pattern is the most specific, highest-confidence
    signal available. Fully rule-based and traceable."""
    features = parse_explanation(explanation)

    if not features:
        return ("T1499 - Network Denial of Service (default)",
                "Unclassified", 0.0, 0.0)

    timing_score = sum(z for f, z in features if f in TIMING_FEATURES)
    volume_score = sum(z for f, z in features if f in VOLUME_FEATURES)
    scan_score = sum(z for f, z in features if f in SCAN_FEATURES)

    if scan_score > 0 and scan_score >= max(timing_score, volume_score):
        return ("T1046 - Network Service Scanning",
                f"High connection/port-diversity burst pattern - scan ")
    elif timing_score > volume_score:
        return ("T1046 - Network Service Scanning",
                f"High connection/port-diversity burst pattern - scan ")
    else:
        return ("T1046 - Network Service Scanning",
                f"High connection/port-diversity burst pattern - scan ")
```

```

        f"evidence ({scan_score:.1f}) dominates.",
        timing_score, volume_score)
    elif timing_score == 0 and volume_score == 0:
        return ("T1499 - Network Denial of Service (general)",
            "Anomalous behavior detected but does not match a "
            "specific known DoS sub-pattern - recommend manual "
            "analyst review.", timing_score, volume_score)
    elif timing_score > volume_score:
        return ("T1499.002 - Endpoint DoS: Service Exhaustion Flood",
            f"Slow-rate resource exhaustion pattern - timing evidence "
            f"({timing_score:.1f}) outweighs volume evidence "
            f"({volume_score:.1f}) across the flagged features.",
            timing_score, volume_score)
    else:
        return ("T1498 - Network Denial of Service",
            f"High-volume flood pattern - volume evidence "
            f"({volume_score:.1f}) outweighs timing evidence "
            f"({timing_score:.1f}) across the flagged features.",
            timing_score, volume_score)

if __name__ == "__main__":
    input_path = sys.argv[1]
    output_path = sys.argv[2]

    print(f"Loading {input_path} ...")
    df = pd.read_csv(input_path)
    print(f"Mapping {len(df)} anomalies to MITRE ATT&CK techniques...")

    mitre_ids, mitre_reasons, timing_scores, volume_scores = [], [], [], []
    for explanation in df["explanation"]:
        technique, reason, t_score, v_score = map_to_mitre(explanation)
        mitre_ids.append(technique)
        mitre_reasons.append(reason)
        timing_scores.append(t_score)
        volume_scores.append(v_score)

    df["mitre_technique"] = mitre_ids
    df["mitre_reasoning"] = mitre_reasons
    df["timing_evidence_score"] = timing_scores
    df["volume_evidence_score"] = volume_scores

    df.to_csv(output_path, index=False)
    print(f"Saved to {output_path}")
    print("\n--- MITRE technique distribution ---")
    print(df["mitre_technique"].value_counts())
    print("\n--- Cross-check: MITRE mapping vs true attack label ---")
    print(pd.crosstab(df["Label"], df["mitre_technique"]))

```

4.7 dashboard/app.py: Interactive Streamlit Dashboard (Final Version)

The final dashboard reflects the combined detection system output. Data source paths point to the combined and final MITRE mapped files, the risk severity table includes the new scanning technique, and the alert table surfaces the scan_flag column so an analyst can see which detector caught each anomaly.

```
import streamlit as st
import pandas as pd
import plotly.express as px

st.set_page_config( page_title="AttackDN
    A Dashboard", page_icon="dna",
    layout="wide",
    initial_sidebar_state="expanded"
)

st.markdown("""
<style>
div[data-testid="stMetric"]
{ background-color:
#1e2130; border: 1px solid
#2d3348; padding: 15px;
border-radius: 8px;
}
.stTabs [data-baseweb="tab-list"] { gap: 24px; }
</style>
""", unsafe_allow_html=True)

TOTAL_ENTITIES_PATH = "data/processed/wednesday_combined.csv"
ALERTS_PATH = "data/processed/wednesday_mitre_final.csv"

# Assign business criticality per host (0-100). Customize to your
# actual network's asset importance - defaults to 50 for anything not
# listed. This is the "Asset Criticality" term in the risk formula.
ENTITY_CRITICALITY = {
    "192.168.10.50": 90,    # Web server - primary DoS target, business-critical
}
DEFAULT_CRITICALITY = 50

MITRE_SEVERITY = {
    "T1498 - Network Denial of Service": 85,
    "T1499.002 - Endpoint DoS: Service Exhaustion Flood": 75,
    "T1046 - Network Service Scanning": 70,
    "T1499 - Network Denial of Service (general)": 50,
    "T1499 - Network Denial of Service (default)": 30,
}

@st.cache_data
def load_total_entity_count():
    """Entities Monitored should reflect everything the system watches,
    not just entities that happen to have a flagged anomaly today."""
    df = pd.read_csv(TOTAL_ENTITIES_PATH, usecols=["entity"])
    return df["entity"].nunique()

@st.cache_data
def load_alerts():
    return pd.read_csv(ALERTS_PATH)
```

```

def compute_risk_score(df, threat_intel_score=0):
    """Weighted risk score: 40% anomaly severity, 30% asset
    criticality, 20% MITRE technique severity, 10% threat
    intelligence (defaults to 0 - no live feed, future work)."""
    df = df.copy()
    min_score, max_score = df["anomaly_score"].min(), df["anomaly_score"].max()
    spread = max_score - min_score
    if spread == 0:
        anomaly_component = pd.Series(50.0, index=df.index)
    else:
        anomaly_component = 100 * (max_score - df["anomaly_score"]) / spread

    criticality_component = df["entity"].map(ENTITY_CRITICALITY).fillna(DEFAULT_CRITICALITY)
    mitre_component = df["mitre_technique"].map(MITRE_SEVERITY).fillna(40)

    df["risk_score"] = (
        0.40 * anomaly_component +
        0.30 * criticality_component +
        0.20 * mitre_component +
        0.10 * threat_intel_score
    )
    return df

def risk_band(score):
    if score <= 30: return "Normal"
    elif score <= 60: return "Suspicious"
    elif score <= 80: return "High"
    return "Critical"

BAND_COLORS = {"Normal": "#2ecc71", "Suspicious": "#f1c40f",
               "High": "#e67e22", "Critical": "#e74c3c"}

total_entities = load_total_entity_count()
df = load_alerts()
df = compute_risk_score(df)
df["risk_band"] = df["risk_score"].apply(risk_band)

st.title("AttackDNA - Behavioral Anomaly Detection")
st.caption("Per-entity Isolation Forest models plus a burst scan detector, "
           "trained on Monday/Tuesday/Thursday benign baselines.")

st.sidebar.header("Filters")
entities = sorted(df["entity"].unique())
selected_entity = st.sidebar.selectbox("Entity", ["All"] + entities)
selected_bands = st.sidebar.multiselect(
    "Risk Band", ["Normal", "Suspicious", "High", "Critical"],
    default=["Suspicious", "High", "Critical"])
selected_techniques = st.sidebar.multiselect(
    "MITRE Technique", sorted(df["mitre_technique"].unique()),
    default=sorted(df["mitre_technique"].unique()))

view_df = df.copy()
if selected_entity != "All":
    view_df = view_df[view_df["entity"] == selected_entity]
if selected_bands:
    view_df = view_df[view_df["risk_band"].isin(selected_bands)]
if selected_techniques:
    view_df = view_df[view_df["mitre_technique"].isin(selected_techniques)]

col1, col2, col3, col4 = st.columns(4)
col1.metric("Entities Monitored", total_entities)

```

```

col2.metric("Flagged Anomalies (filtered)", f"{len(view_df):}")
col3.metric("Critical Risk Alerts", f"{{(view_df['risk_band'] == 'Critical').sum():}}")
col4.metric("MITRE Techniques Seen", view_df["mitre_technique"].nunique())

st.divider()
tab1, tab2, tab3 = st.tabs(["Overview", "Top Entities", "Alert Table"])

with tab1:
    col_a, col_b = st.columns(2)
    with col_a:
        st.subheader("Risk Band Distribution")
        band_counts = view_df["risk_band"].value_counts().reindex(
            ["Normal", "Suspicious", "High", "Critical"]).fillna(0).reset_index()
        band_counts.columns = ["Risk Band", "Count"]
        fig = px.bar(band_counts, x="Risk Band", y="Count", color="Risk Band",
                     color_discrete_map=BAND_COLORS)
        fig.update_layout(showlegend=False)
        st.plotly_chart(fig, use_container_width=True)
    with col_b:
        st.subheader("MITRE Technique Distribution")
        mitre_counts = view_df["mitre_technique"].value_counts().reset_index()
        mitre_counts.columns = ["Technique", "Count"]
        fig = px.bar(mitre_counts, x="Count", y="Technique", orientation="h")
        st.plotly_chart(fig, use_container_width=True)

    st.subheader("True Label Breakdown")
    st.caption("Ground truth from CICIDS2017 - for validation only, not visible in a real
    deployment.")
    label_counts = view_df["Label"].value_counts().reset_index()
    label_counts.columns = ["Label", "Count"]
    fig = px.pie(label_counts, names="Label", values="Count", hole=0.4)
    st.plotly_chart(fig, use_container_width=True)

with tab2:
    st.subheader("Entities Ranked by Average Risk Score")
    entity_summary = (view_df.groupby("entity")
        .agg(avg_risk=("risk_score", "mean"), alerts=("risk_score", "count"))
        .sort_values("avg_risk", ascending=False).reset_index())
    fig = px.bar(entity_summary, x="entity", y="avg_risk", color="avg_risk",
                 color_continuous_scale="Reds", labels={"avg_risk": "Avg Risk Score"})
    st.plotly_chart(fig, use_container_width=True)
    st.dataframe(entity_summary, use_container_width=True)

with tab3:
    st.subheader("Flagged Alerts (highest risk first)")
    display_cols = ["entity", "risk_score", "risk_band", "Label",
                    "mitre_technique", "mitre_reasoning", "scan_flag", "explanation"]
    display_df = view_df.sort_values("risk_score", ascending=False)[display_cols].head(300)

    def highlight_band(row):
        color = BAND_COLORS.get(row["risk_band"], "white")
        return [f"background-color: {color}20" * len(row)]

    st.dataframe(
        display_df.style.apply(highlight_band, axis=1).format({"risk_score": "{:.1f}"}),
        use_container_width=True, height=550)

    st.download_button("Download filtered alerts as CSV",
        data=view_df.to_csv(index=False).encode("utf-8"),
        file_name="attackdna_filtered_alerts.csv", mime="text/csv")

st.caption("Risk score = 40% anomaly severity + 30% asset criticality + "
    "20% MITRE severity + 10% threat intelligence (no live feed configured).")

```


5 Initial Results, Before Supervisor Review

The first full pipeline run trained 15 per entity models on Monday's benign traffic only, then evaluated against Wednesday's DoS traffic, namely Hulk, GoldenEye, Slowloris, Slowhttptest, and Heartbleed.

5.1 Detection Rate by True Label

True Label	Flagged / Total	Detection Rate
BENIGN	36,111 / 648,986	5.6% (false positive rate)
DoS Hulk	153,905 / 230,123	66.9%
DoS GoldenEye	6,779 / 10,293	65.9%
DoS slowloris	3,012 / 5,796	52.0%
DoS Slowhttptest	4,005 / 5,499	72.8%
Heartbleed	11 / 11	100.0%

This result followed a threshold calibration fix. The initial version used Isolation Forest's default contamination='auto' setting, which flags roughly 10 percent of any dataset regardless of how clean it actually is. This was diagnosed by observing an inflated 8.8 percent network wide false positive rate, and fixed by calibrating a custom per entity threshold on a held out validation split of each entity's own baseline, targeting a 5 percent false positive rate. This reduced network false positive rate from 8.8 percent to 5.6 percent.

5.2 The 192.168.10.50 Outlier

Entity 192.168.10.50, the CICIDS2017 web server and primary DoS target, showed an elevated false positive rate of 18.5 percent compared to the network average of 5.6 percent, even after threshold calibration. Investigation confirmed this entity had the second smallest Monday baseline, 11,307 flows, and was the sole target of sustained DoS attacks throughout Wednesday. This was documented as a known limitation of per entity modeling and became the starting point for the baseline expansion work summarized in Section 6.

6 Limitations Identified and Addressed

Following the initial build, twelve specific limitations were identified through structured review. Each was investigated with real data rather than assumed. The table below summarizes the status of all twelve; the most significant items are discussed in more detail afterward.

#	Limitation	Status
1	Limited baseline data, one day only	Addressed, expanded to four days, mixed result
2	Static baseline, never updates	Addressed by design, retraining process defined
3	Same threshold for every host	Addressed, calibrated per entity, trade off documented
4	Only one ML algorithm tested	Addressed in Section 7.2
5	No feature selection	Addressed, 79 to 57 features
6	Weak MITRE mapping logic	Addressed, evidence based scoring across all features

#	Limitation	Status
7	Explanations are statistical only	Partially addressed, model native explanations remain future work
8	No real time detection	Documented as future work
9	Tested on only one day, one attack type	Addressed, expanded to two days, five attack types
10	Risk score used only one factor	Addressed, weighted four factor score
11	No multi flow pattern detection	Proven limitation, directly addressed in Section 7.1
12	No formal accuracy metrics	Addressed, precision, recall, F1 added

6.1 Baseline Expansion (Item 1)

The model was originally trained on Monday only. train.py was rewritten to accept multiple CSV files, filter each to Label == 'BENIGN' only, and concatenate them. The combined baseline grew to 2,106,675 rows across all 15 entities. The result was mixed: network wide false positive rate barely changed, from 5.6 to 6.0 percent, and entity .50's false positive rate actually rose slightly, from 18.5 to 21.8 percent. This is genuine evidence that adding more historical days of data does not fully close the gap to an unseen test day, and is connected to the static baseline limitation below.

6.2 Threshold Calibration Comparison (Item 3)

Two threshold calibration approaches were implemented and compared: the fixed 5 percent target percentile method already described, and an adaptive k sigma method using the mean minus three standard deviations of decision scores, computed per entity.

Metric	Fixed 5% target	Adaptive k=3 σ
Precision	0.815	0.872
Recall	0.683	0.612
F1	0.743	0.719
FPR	0.060	0.035

The fixed 5 percent target method wins on F1 and recall, meaning fewer missed attacks. The adaptive k sigma method wins on precision and false positive rate, meaning fewer false alarms. This is a genuine operational trade off between analyst alert fatigue and missed attack risk, not a case of one method being simply better.

A bug was found and fixed during this work. An intermediate version of train.py grouped by the raw Source IP column instead of the pre computed entity column, silently truncating each host's training data to only flows where it happened to initiate the connection. Entity 192.168.10.1, for example, dropped to just two training rows and was skipped entirely. This was fixed by grouping on entity consistently throughout the pipeline.

6.3 Rule-Based MITRE Mapping Improvement (Item 6)

The original mapping rule looked only at the single highest deviation feature in each anomaly's explanation. This was rewritten to parse all features mentioned in the explanation, sum z-score evidence per category, and classify based on which category's total evidence dominates.

Attack Type	Correctly Mapped	Notes
DoS slowloris	3,283 / 3,287	99.9%, timing category
DoS Slowhttptest	4,009 / 4,029	99.5%, timing category
DoS GoldenEye	7,265 / 8,657	approximately 84%, volume category
DoS Hulk	split	roughly 53% / 47% between categories, honest limitation

6.4 Extended Evaluation (Item 9)

The original evaluation covered only Monday to Wednesday, a single attack family. Three further held out test files were added, namely Friday Morning for Bot traffic, Friday Afternoon PortScan, and Friday Afternoon DDoS.

Attack Type (Day)	Precision	Recall	F1
DoS (Wednesday)	0.973	0.683	0.803
DDoS (Friday)	0.836	0.636	0.708
Bot (Friday)	approx. 0.00	0.022	approx. 0.00
PortScan (Friday)	0.521	0.002	0.003

This was the most important finding of the initial remediation phase: Bot and PortScan recall are both effectively zero. Investigating this gap directly led to the supervisor's highest priority request, covered next.

7 Supervisor-Requested Improvements

After reviewing the results above, the project supervisor requested four specific follow up items, in priority order: add time window features to close the Bot and PortScan detection gap, benchmark a second algorithm, Local Outlier Factor, for comparison, validate results across multiple data splits to confirm they are not a fluke, and diagnose the root cause of the roughly 5.6 percent false positive rate.

7.1 Time Window Features (Highest Priority)

7.1.1 Motivation

Prior evaluation had shown that the per flow Isolation Forest model achieved only 0.2 percent recall on PortScan and 2.2 percent recall on Bot traffic, versus 63 to 84 percent recall on DoS and DDoS. The diagnosis was that PortScan and Bot attacks do not produce statistically extreme individual flows. A single port scan probe, one small packet, one quick connection, looks unremarkable in isolation. The anomalous signal only exists when looking at multiple flows from the same host over a short time window, for example one host opening hundreds of connections in a few seconds. Since the model scored each flow independently, this pattern was invisible to it by construction, not because of a tuning problem.

7.1.2 Design and Implementation

Two new engineered features were added: `connections_per_minute`, the count of flows generated by the same entity within the same 60 second time bucket, and `distinct_ports_per_minute`, the count of unique destination ports touched by the same entity within that same bucket. The implementation uses fixed 60 second buckets, computed per entity, then merged back onto the original per flow data so every flow inherits its bucket's aggregate counts.

A limitation of this design is stated explicitly rather than hidden: fixed boundary bucketing has an edge effect, where an attack burst that straddles a minute boundary is split into two smaller counts instead of registering as one large burst. A true sliding window would avoid this but is more complex to implement, so fixed buckets were chosen as a reasonable, explainable trade off given the project timeline.

```
import pandas as pd
import numpy as np
import sys

def add_time_window_features(df, window="60s"):
    """Add two time-window features per flow: how many connections this
    entity made, and how many distinct destination ports it touched,
    in the same 60-second bucket as this flow. Computed per-entity so
    one host's burst doesn't inflate another host's numbers."""
    df = df.copy()
    df["_ts"] = pd.to_datetime(df["Timestamp"], errors="coerce")

    n_bad = df["_ts"].isna().sum()
    if n_bad > 0:
        print(f" Warning: {n_bad} rows had unparseable timestamps "
              f"(features will be NaN -> filled with 0 for these)")

    df["_bucket"] = df["_ts"].dt.floor(window)

    conn_counts = (
        df.groupby(["entity", "_bucket"])
        .size()
        .rename("connections_per_minute")
```

```

        .reset_index()
    )
    port_counts = (
        df.groupby(["entity", "_bucket"])["Destination Port"]
        .nunique()
        .rename("distinct_ports_per_minute")
        .reset_index()
    )

    df = df.merge(conn_counts, on=["entity", "_bucket"], how="left")
    df = df.merge(port_counts, on=["entity", "_bucket"], how="left")

    df["connections_per_minute"] = df["connections_per_minute"].fillna(0)
    df["distinct_ports_per_minute"] = df["distinct_ports_per_minute"].fillna(0)

    df = df.drop(columns=["_ts", "_bucket"])
    return df

if __name__ == "__main__":
    input_path = sys.argv[1]
    output_path = sys.argv[2]

    print(f"Loading {input_path} ...")
    df = pd.read_csv(input_path)
    print(f"Loaded {len(df)} rows.")

    df = add_time_window_features(df)

    print("\n--- New feature summary ---")
    print(df[["connections_per_minute", "distinct_ports_per_minute"]].describe())

    df.to_csv(output_path, index=False)
    print(f"\nSaved to {output_path}")

```

This was applied to all eight processed data files, the four baseline days and the four test days, producing a `tw` version of each.

7.1.3 Verification Before Retraining

Before retraining, the raw feature separation was checked directly. Network wide, the mean `connections_per_minute` was 400 to 930 on benign baseline days, 3,915 on Wednesday during DoS, 15,586 on Friday PortScan, roughly 30 times the baseline, and 5,109 on Friday DDoS.

An entity level check specifically for 192.168.10.50, the host targeted by both DoS and PortScan, used the following diagnostic script.

```

import pandas as pd
import sys

entity = "192.168.10.50"

baseline_files =
    [ "data/processed/monday_tw.csv",
      "data/processed/tuesday_tw.csv",
      "data/processed/thursday_morning_tw.csv",
      "data/processed/thursday_afternoon_tw.csv",
    ]
attack_file = "data/processed/friday_portscan_tw.csv"

print(f"--- {entity} BASELINE (training, BENIGN only) ---")
baseline = pd.concat([pd.read_csv(f) for f in baseline_files], ignore_index=True)

```

```
baseline = baseline[(baseline["entity"] == entity) & (baseline["Label"] == "BENIGN")]
print(baseline[["connections_per_minute", "distinct_ports_per_minute"]].describe())

print(f"\n--- {entity} DURING PORTSCAN ATTACK ---")
attack = pd.read_csv(attack_file)
attack_entity = attack[attack["entity"] == entity]
print(attack_entity.groupby("Label")[["connections_per_minute", "distinct_ports_per_minute"]].describe())
```

This confirmed baseline connections_per_minute had a mean of 200 and standard deviation of 363, while during the PortScan attack the mean rose to 33,478 with a standard deviation of 15,394, roughly 90 standard deviations from baseline. The distinct_ports_per_minute baseline median of 15 rose to an attack median of 1,047. This confirmed an extremely strong, unambiguous statistical signal existed in the raw data before any model was involved.

7.1.4 First Attempt: Retraining Isolation Forest Directly Did Not Work

Retraining train.py on the tw files, 59 total features up from 57, and re-evaluating produced no meaningful change. PortScan recall stayed at 0.2 percent, Bot recall stayed at 2.2 percent, despite the massive underlying signal just confirmed. DoS and DDoS numbers also stayed essentially flat.

7.1.5 Root Cause Diagnosis

This is a genuine, mechanistic finding, not a tuning failure. Isolation Forest detects anomalies by measuring how few random splits are needed to isolate a point. Genuine outliers are rare and scattered, so they isolate quickly, giving a short average path length and a high anomaly score. However, the 158,804 PortScan flows in the test file share nearly identical connections_per_minute and distinct_ports_per_minute values, because they are all aggregated from the same handful of one minute buckets during the attack. This is not a handful of scattered outlier points, it is a large, dense cluster of near identical values, far from normal traffic but tightly packed together internally.

Isolation Forest can separate this cluster from normal traffic in one or two splits, but then still has to keep splitting within the cluster to isolate each of the 158,804 near identical points from each other. This takes many additional splits, producing a longer average path length and therefore a lower anomaly score, even though the traffic is obviously abnormal from a security standpoint. This is a documented, known structural weakness of Isolation Forest: it is designed for scattered point anomalies, not for anomalies that form their own dense, self similar cluster, which is exactly what a scan or flood produces.

7.1.6 Fix: A Dedicated Rule-Based Scan Detector

Rather than trying to force Isolation Forest to handle this pattern, a simple, targeted statistical detector was built specifically for burst and volumetric patterns. It computes the per entity mean and standard deviation of the two time window features using only benign baseline traffic, then flags a flow if either metric exceeds mean plus five standard deviations, intentionally stricter than the $k=3$ used elsewhere, since this is a narrowly targeted, high confidence check.

```
import pandas as pd
import numpy as np
import sys

def build_scan_thresholds(baseline_df, k_sigma=5.0):
    """Per-entity mean+std of the two time-window features, using
    ONLY benign baseline traffic. k=5 is intentionally stricter than
    the main model's k=3, since this check targets a specific,
```

```

high-confidence signal rather than general anomalies."
stats =
    baseline_df.groupby("entity")[["connections_per_minute",
    "distinct_ports_per_minute"]
    ].agg(["mean", "std"])
stats.columns = ["_".join(c) for c in stats.columns]

stats["conn_threshold"] = stats["connections_per_minute_mean"] + k_sigma * stats["
connections_per_minute_std"]
stats["port_threshold"] = stats["distinct_ports_per_minute_mean"] + k_sigma * stats["
distinct_ports_per_minute_std"]
return stats[["conn_threshold", "port_threshold"]]

def flag_scans(df, thresholds):
    df = df.copy()
    df = df.merge(thresholds, on="entity", how="left")
    df["scan_flag"] = (
        (df["connections_per_minute"] > df["conn_threshold"]) |
        (df["distinct_ports_per_minute"] > df["port_threshold"])
    )
    return df

if __name__ == "__main__":
    baseline_paths = sys.argv[1].split(",")
    test_path = sys.argv[2]
    output_path = sys.argv[3]

    baseline = pd.concat([pd.read_csv(p) for p in baseline_paths], ignore_index=True)
    baseline = baseline[baseline["Label"] == "BENIGN"]

    thresholds = build_scan_thresholds(baseline)
    print(thresholds)

    df = pd.read_csv(test_path)
    df = flag_scans(df, thresholds)

    print("\n--- Scan-flag results by true label ---")
    for label, group in df.groupby("Label"):
        flagged = group["scan_flag"].sum()
        print(f"{label}: {flagged}/{len(group)} flagged ({flagged/len(group):.1%})")

    df.to_csv(output_path, index=False)

```

7.1.7 Scan Detector Results (Standalone)

Metric	Value
PortScan recall	99.1% (157,441 / 158,804)
PortScan false positive rate	0.7% (1,359 / 190,242 benign flows)
Bot recall	0% (0 / 1,956), expected, see below

PortScan is a burst attack, many connections, many ports, a very short window, which is exactly what these two features are built to catch. Bot and command and control traffic is deliberately the opposite: quiet, low volume, periodic check in beacons specifically designed to avoid looking like a burst, for example one small connection every 60 seconds. This is invisible to a volume based feature by the fundamental nature of the attack, not a flaw in the fix. Catching Bot traffic requires a different signal entirely, beaconing regularity, meaning whether an entity

contacts the same destination at suspiciously fixed time intervals, which is a separate detector explicitly scoped as future work.

7.1.8 Combining the Scan Detector with Isolation Forest

The final flag logic is: a flow is flagged if Isolation Forest predicted an anomaly, or if the scan detector flagged it, whichever is true.

An implementation bug was found and fixed while combining the two detectors. The first attempt used Flow ID plus entity as the join key between the two scored files. This produced nonsensical results, with precision and recall both around 0.12, far below either detector's individual performance, because CICIDS2017's Flow ID is not actually unique. It is built from a five tuple of source and destination IP, source and destination port, and protocol, plus a coarse timestamp, so multiple genuinely different flows can share an identical Flow ID string. A merge on a non unique key silently creates a many to many join, multiplying rows instead of matching them one to one, which was confirmed by checking that true positives plus false positives plus false negatives plus true negatives summed to more than the actual file's row count.

The fix was to add a stable, guaranteed unique row_id column, using range(len(df)), to every file at the earliest pipeline stage, in data_prep.py right after entity extraction, and to re-run the entire pipeline from that point forward so every downstream file carries a trustworthy join key. After the fix, the four confusion matrix counts were checked against each file's actual row count as a sanity check before trusting any combined result.

```
import pandas as pd
import numpy as np
import sys

if __name__ == '__main__':
    if_path = sys.argv[1]
    scan_path = sys.argv[2]
    output_path = sys.argv[3]

    if_df = pd.read_csv(if_path)
    scan_df = pd.read_csv(scan_path)[["row_id", "scan_flag"]]
    df = if_df.merge(scan_df, on="row_id", how="left")
    if df["scan_flag"].isna().any():
        print(f"Warning: {df['scan_flag'].isna().sum()} rows had no scan_flag match")
    df["scan_flag"] = df["scan_flag"].fillna(False)

    df["final_flag"] = (df["predicted"] == -1) | (df["scan_flag"] == True)
    df["predicted"] = np.where(df["final_flag"], -1, 1)

    is_attack = df["Label"] != "BENIGN"
    tp = (is_attack & df["final_flag"]).sum()
    fn = (is_attack & ~df["final_flag"]).sum()
    fp = (~is_attack & df["final_flag"]).sum()
    tn = (~is_attack & ~df["final_flag"]).sum()

    precision = tp / (tp + fp) if (tp + fp) else 0
    recall = tp / (tp + fn) if (tp + fn) else 0
    f1 = 2 * precision * recall / (precision + recall) if (precision + recall) else 0

    print(f"Combined system: TP={tp} FP={fp} FN={fn} TN={tn}")
    print(f"Precision: {precision:.3f} Recall: {recall:.3f} F1: {f1:.3f}")

    df.to_csv(output_path, index=False)
```

Note that this final version of combine_detectors.py also writes the combined result back into the standard predicted column. This means explain.py and mitre_map.py, which both key

off predicted == -1, automatically treat scan detector catches identically to Isolation Forest catches with no additional code changes needed in those scripts.

7.1.9 Final Combined System Results

Attack	IF alone Recall / F1	Combined Recall / F1
DoS (Wednesday)	68.3% / 0.743	98.1% / 0.913
DDoS (Friday)	63.6% / 0.722	99.3% / 0.872
PortScan (Friday)	0.2% / 0.003	99.2% / 0.962
Bot (Friday)	2.2% / 0.005	2.2% / 0.005, unchanged

An interesting side finding is that the scan detector also caught 99.8 percent of DoS Hulk and 39.5 percent of DoS GoldenEye on its own, since both are high request rate floods that also trip the burst thresholds, meaning the fix improved DoS and DDoS detection too, not only PortScan.

An honest trade off must be reported: on the DDoS test day, the combined system's false positive rate rose to 27.6 percent, up from single digits with Isolation Forest alone, because the scan detector's five sigma threshold gets tripped by naturally chaotic background traffic on an already bursty attack day. Precision dropped from 0.836, Isolation Forest alone, to 0.777, combined, on this day. This is still a clear net win on F1, from 0.722 to 0.872, but the cost should be stated plainly and not hidden.

7.2 Local Outlier Factor Comparison

7.2.1 Motivation and Conceptual Difference

This item tested whether a fundamentally different anomaly detection algorithm handles the dataset, and specifically the dense cluster problem diagnosed above, differently than Isolation Forest. Isolation Forest asks how easily a point can be isolated by random splits. Local Outlier Factor instead asks how much sparser the local neighborhood around a point is compared to its neighbors' neighborhoods, comparing each point's local density, via k nearest neighbors, to the local density of its neighbors. A point sitting in a much sparser region than its neighbors gets a high outlier score. This is a local, density based comparison rather than a global tree splitting approach, which was hypothesized to handle dense clustered anomalies, such as scan bursts, differently.

7.2.2 Implementation

`sklearn.neighbors.LocalOutlierFactor` was used with `novelty=True`, required to score new, unseen test data, since LOF is unsupervised by default and cannot otherwise score points outside its training set. The same per entity training structure, the same five percent target false positive rate percentile calibration, and the same train and validation split methodology were used as for Isolation Forest, for a fair comparison. Training rows were capped at 20,000 per entity, down from up to 597,000 for the busiest entity, since LOF's memory and time cost scales faster than Isolation Forest's with training set size. This is a documented, deliberate trade off, not an oversight, and training completed in 45.9 seconds for all 15 entities, confirming it was not a performance bottleneck at this cap. The number of neighbors was set to 20, the standard default, reduced automatically for entities with fewer than 21 training rows.

```
import sys
import pandas as pd
import numpy as np
import pickle
import os
import time
from sklearn.neighbors import LocalOutlierFactor
from sklearn.preprocessing import StandardScaler

NON_FEATURE_COLUMNS = [
    "Flow ID", "Source IP", "Destination IP", "Timestamp",
    "Source Port", "Label", "entity", "row_id"
]

def get_feature_columns(df):
    return [col for col in df.columns if col not in NON_FEATURE_COLUMNS]

def train_entity_lof(entity_df, feature_cols, min_flows=100, target_fpr=0.05,
                    n_neighbors=20, max_train_rows=20000):
    """Train one LOF model per entity, same target-FPR calibration
    approach as the Isolation Forest version. LOF training data is
    capped at max_train_rows for very large entities, since LOF's
    memory/time cost grows fast with training set size - this is a
    documented, deliberate trade-off, not an oversight."""
    if len(entity_df) < min_flows:
        print(f" Skipping - only {len(entity_df)} flows (need {min_flows}).")
        return None, None, None

    shuffled = entity_df.sample(frac=1, random_state=42)
    if len(shuffled) > max_train_rows:
```

```

        print(f" Capping training rows: {len(shuffled)} -> {max_train_rows}")
        shuffled = shuffled.iloc[:max_train_rows]

        split_point = int(len(shuffled) * 0.8)
        train_df = shuffled.iloc[:split_point]
        val_df = shuffled.iloc[split_point:]

        X_train = train_df[feature_cols].values
        X_val = val_df[feature_cols].values

        scaler = StandardScaler()
        X_train_scaled = scaler.fit_transform(X_train)
        X_val_scaled = scaler.transform(X_val)

        model = LocalOutlierFactor(
            n_neighbors=min(n_neighbors, len(X_train_scaled) - 1),
            novelty=True,
            n_jobs=-1
        )
        model.fit(X_train_scaled)

        val_scores = model.decision_function(X_val_scaled)
        threshold = np.percentile(val_scores, target_fpr * 100)

        return model, scaler, threshold

def save_entity_model(entity_ip, model, scaler, threshold, feature_cols, out_dir="models_lof")
:
    os.makedirs(out_dir, exist_ok=True)
    safe_name = entity_ip.replace(".", "_")
    path = os.path.join(out_dir, f"{safe_name}.pkl")

    bundle = {
        "model": model, "scaler": scaler, "threshold": threshold,
        "feature_cols": feature_cols, "entity_ip": entity_ip
    }
    with open(path, "wb") as f:
        pickle.dump(bundle, f)
    print(f" Saved -> {path} (threshold={threshold:.4f})")

if __name__ == "__main__":
    input_paths = sys.argv[1:] if len(sys.argv) > 1 else ["data/processed/monday_tw.csv"]

    print(f"Loading baseline from {len(input_paths)} file(s)...")
    frames = []
    for path in input_paths:
        day_df = pd.read_csv(path)
        day_df = day_df[day_df["Label"] == "BENIGN"]
        frames.append(day_df)
    df = pd.concat(frames, ignore_index=True)
    print(f"Combined baseline: {len(df)} BENIGN rows, {df['entity'].nunique()} entities\n")

    feature_cols = get_feature_columns(df)
    print(f"Using {len(feature_cols)} feature columns.")

    entities = df["entity"].unique()
    print(f"Training LOF models for {len(entities)} entities...\n")

    start = time.time()
    trained_count = skipped_count = 0

```

```

for entity_ip in entities:
    print(f"Entity: {entity_ip} ({len(df[df['entity']==entity_ip])} rows)")
    entity_df = df[df["entity"] == entity_ip]
    model, scaler, threshold = train_entity_lof(entity_df, feature_cols)
    if model is None:
        skipped_count += 1
        continue
    save_entity_model(entity_ip, model, scaler, threshold, feature_cols)
    trained_count += 1

elapsed = time.time() - start
print(f"\nDone. Trained: {trained_count}, Skipped: {skipped_count}")
print(f"Total training time: {elapsed:.1f}s")

```

```

import pandas as pd
import numpy as np
import pickle
import os
import sys
import time

def load_entity_model(entity_ip, models_dir="models_lof"):
    safe_name = entity_ip.replace(".", "_")
    path = os.path.join(models_dir, f"{safe_name}.pkl")
    if not os.path.exists(path):
        return None
    with open(path, "rb") as f:
        return pickle.load(f)

def score_entity(entity_df, bundle):
    model, scaler, threshold = bundle["model"], bundle["scaler"], bundle["threshold"]
    feature_cols = bundle["feature_cols"]
    X = entity_df[feature_cols].values
    X_scaled = scaler.transform(X)
    anomaly_score = model.decision_function(X_scaled)
    predicted = np.where(anomaly_score < threshold, -1, 1)
    entity_df = entity_df.copy()
    entity_df["predicted"] = predicted
    entity_df["anomaly_score"] = anomaly_score
    return entity_df

if __name__ == "__main__":
    input_path = sys.argv[1]
    output_path = sys.argv[2] if len(sys.argv) > 2 else "data/processed/lof_scored.csv"

    print(f"Loading {input_path} ...")
    df = pd.read_csv(input_path)
    print(f"Loaded {len(df)} rows.")

    entities = df["entity"].unique()
    print(f"Scoring {len(entities)} entities with LOF...\n")

    results, no_model_count = [], 0
    start = time.time()

    for entity_ip in entities:
        bundle = load_entity_model(entity_ip)
        if bundle is None:
            print(f"Entity {entity_ip}: no LOF model found, skipping.")

```

```

        no_model_count += len(df[df["entity"] == entity_ip])
        continue
    entity_df = df[df["entity"] == entity_ip]
    scored_df = score_entity(entity_df, bundle)
    results.append(scored_df)
    n_anomalies = (scored_df["predicted"] == -1).sum()
    print(f"Entity {entity_ip}: {len(scored_df)} flows, {n_anomalies} flagged.")

all_results = pd.concat(results, ignore_index=True)
elapsed = time.time() - start
print(f"\nScoring time: {elapsed:.1f}s")
print(f"Rows skipped (no model): {no_model_count}")

print("\n--- Detection results by true label ---")
summary = all_results.groupby("Label")["predicted"].apply(lambda x: (x == -1).sum())
total_by_label = all_results["Label"].value_counts()
for label in total_by_label.index:
    flagged = summary.get(label, 0)
    total = total_by_label[label]
    print(f"{label}: {flagged}/{total} flagged as anomaly ({flagged/total:.1%})")

all_results.to_csv(output_path, index=False)
print(f"\nSaved to {output_path}")

```

7.2.3 Results: LOF versus Isolation Forest

Attack	IF alone F1	LOF alone F1
DoS (Wednesday)	0.743	0.828
DDoS (Friday)	0.722	0.855
PortScan (Friday)	0.003	0.957
Bot (Friday)	approx. 0.00	0.013

Full LOF numbers, network wide:

Attack	Precision	Recall	F1	FPR
DoS (Wed)	0.801	0.857	0.828	0.083
DDoS (Fri)	0.782	0.942	0.855	0.254
PortScan (Fri)	0.923	0.994	0.957	0.069
Bot (Fri)	0.007	0.064	0.013	0.061

LOF outperforms Isolation Forest on every single attack type tested, not only PortScan. Most notably, LOF natively achieves 99.4 percent PortScan recall with zero additional detector layered on, nearly matching the hand engineered Isolation Forest plus scan detector combined system's 99.2 percent recall, using a single model. This directly confirms the earlier mechanistic diagnosis: LOF's local density comparison naturally handles dense clustered anomalies, such as scan bursts, in a way that Isolation Forest's global tree splitting approach cannot, without needing any manual feature threshold rule.

Bot traffic remains essentially undetected across all three approaches tested, Isolation Forest at approximately 0 percent, LOF at 6.4 percent recall, and Isolation Forest plus scan detector at approximately 0 percent. This is a triple confirmed finding that Bot detection requires a fundamentally different signal, beaconing periodicity analysis, rather than a different anomaly algorithm or a burst based rule.

Recommended framing: the two algorithms were compared head to head under identical calibration methodology. LOF outperforms Isolation Forest on every attack type tested and largely closes the PortScan detection gap on its own, without requiring the manually engineered scan detector rule, though the Isolation Forest plus scan detector combined system still edges out LOF alone on DoS and DDoS, 0.913 and 0.872 versus 0.828 and 0.855, since the scan detector layer catches burst patterns on top of whatever the base model already catches, regardless of which core algorithm is used underneath. A natural next experiment, not yet run, is testing LOF together with the same scan detector layered on top.

7.3 Split and Seed Stability Testing

7.3.1 Motivation

All prior results relied on a single random 80/20 train and validation split, with a fixed random state of 42, inside each entity's training process. A single split could, by chance, be unusually favorable or unfavorable, which would make reported numbers unreliable indicators of true performance.

7.3.2 Method

A script was built that repeats the full training and evaluation process for Isolation Forest with five different random seeds, namely 1, 7, 21, 99, and 123, entirely in memory with no intermediate files, and reports the mean and standard deviation of precision, recall, and F1 across the five runs. This was applied to three test days: Wednesday for DoS, Friday DDoS, and Friday PortScan.

```
import sys
import pandas as pd
import numpy as np
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler

NON_FEATURE_COLUMNS = [
    "Flow ID", "Source IP", "Destination IP", "Timestamp",
    "Source Port", "Label", "entity", "row_id"
]

def get_feature_columns(df):
    return [col for col in df.columns if col not in NON_FEATURE_COLUMNS]

def train_and_score(baseline_df, test_df, feature_cols, seed, min_flows=100, target_fpr=0.05):
    """Train all entities with the given random seed, then score the
    test set and return network-wide TP/FP/FN/TN. Nothing is saved to
    disk - this is purely for measuring seed-to-seed stability."""
    all_predictions = []
    for entity_ip in baseline_df["entity"].unique():
        entity_df = baseline_df[baseline_df["entity"] == entity_ip]
        if len(entity_df) < min_flows:
            continue
        shuffled = entity_df.sample(frac=1, random_state=seed)
        split_point = int(len(shuffled) * 0.8)
        train_part = shuffled.iloc[:split_point]
        val_part = shuffled.iloc[split_point:]
        scaler = StandardScaler()
        X_train = scaler.fit_transform(train_part[feature_cols].values)
        X_val = scaler.transform(val_part[feature_cols].values)
        model = IsolationForest(n_estimators=100, contamination="auto",
                                random_state=seed, n_jobs=-1)
        model.fit(X_train)
        val_scores = model.decision_function(X_val)
        threshold = np.percentile(val_scores, target_fpr * 100)
        test_entity = test_df[test_df["entity"] == entity_ip]
        if len(test_entity) == 0:
            continue
        X_test = scaler.transform(test_entity[feature_cols].values)
        test_scores = model.decision_function(X_test)
        predicted = np.where(test_scores < threshold, -1, 1)
        result = test_entity[["Label"]].copy()
        result["predicted"] = predicted
        all_predictions.append(result)
    all_df = pd.concat(all_predictions, ignore_index=True)
```

```

is_attack = all_df["Label"] != "BENIGN"
is_flagged = all_df["predicted"] == -1
tp = (is_attack & is_flagged).sum()
fn = (is_attack & ~is_flagged).sum()
fp = (~is_attack & is_flagged).sum()
tn = (~is_attack & ~is_flagged).sum()
precision = tp / (tp + fp) if (tp + fp) else 0
recall = tp / (tp + fn) if (tp + fn) else 0
f1 = 2 * precision * recall / (precision + recall) if (precision + recall) else 0
return precision, recall, f1

if __name__ == "__main__":
    baseline_paths = sys.argv[1].split(",")
    test_path = sys.argv[2]
    seeds = [1, 7, 21, 99, 123]
    print(f"Loading baseline from {len(baseline_paths)} file(s)...")
    frames = [pd.read_csv(p) for p in baseline_paths]
    baseline_df = pd.concat(frames, ignore_index=True)
    baseline_df = baseline_df[baseline_df["Label"] == "BENIGN"]
    print(f"Baseline: {len(baseline_df)} BENIGN rows")
    print(f"Loading test file: {test_path}")
    test_df = pd.read_csv(test_path)
    print(f"Test: {len(test_df)} rows\n")
    feature_cols = get_feature_columns(baseline_df)
    results = []
    for seed in seeds:
        print(f"Training with seed={seed}...")
        precision, recall, f1 = train_and_score(baseline_df, test_df, feature_cols, seed)
        print(f" Precision={precision:.3f} Recall={recall:.3f} F1={f1:.3f}")
        results.append({"seed": seed, "precision": precision, "recall": recall, "f1": f1})
    results_df = pd.DataFrame(results)
    print("\n--- Stability across seeds ---")
    print(results_df.to_string(index=False))
    print(f"\nPrecision: mean={results_df['precision'].mean():.3f}
std={results_df['precision'].std():.4f}")
    print(f"Recall: mean={results_df['recall'].mean():.3f} std={results_df['recall'].std()
:.4f}")
    print(f"F1: mean={results_df['f1'].mean():.3f} std={results_df['f1'].std():.4f}")

```

7.3.3 Results

Test day	Mean F1	Std Dev	Relative variation
Wednesday (DoS)	0.745	0.0015	0.2%
Friday DDoS	0.702	0.0052	0.7%
Friday PortScan	0.004	0.0004	approximately 10%

PortScan's standard deviation looks proportionally large only because the F1 value itself is near zero. In absolute terms 0.0004 is negligible, and the practical conclusion, that the base Isolation Forest model essentially never catches PortScan regardless of split, is unchanged across every seed tested.

7.3.4 Interpretation

Isolation Forest results are highly stable across random splits, under 1 percent relative variation in F1 for DoS and DDoS. This makes mechanistic sense: with 2.1 million baseline rows spread across 15 entities, even the smallest entity has thousands of training rows, at which scale which specific rows land in train versus validation has little effect on what the model learns about normal traffic.

This same stability check also proves the PortScan failure is not a fluke either. Isolation Forest reliably fails to detect PortScan across every tested seed, with F1 pinned at approximately 0.004 every time, giving the same statistical rigor to the negative finding as to the positive ones. This also indirectly supports the earlier baseline expansion work, since a small baseline, such as the original Monday only 11,000 row baseline for entity .50, would likely show meaningfully more seed to seed variance than the current large, stable baseline does, though this specific small versus large baseline stability comparison was not directly tested and would need to be run to state as a confirmed result rather than a reasonable inference.

As a scope note, the scan detector's per entity five sigma thresholds involve no randomness, since they are a direct mean and standard deviation calculation from the data, so there is nothing to seed test there. LOF split stability was also not tested, only Isolation Forest.

7.4 Root-Cause Diagnosis of the False Positive Rate

7.4.1 Motivation

The supervisor specifically asked what is causing the baseline false positive rate, rather than treating it as unexplained background noise.

7.4.2 Method

A diagnostic script was built to compute per entity false positive rate, defined as false positives divided by the sum of false positives and true negatives restricted to BENIGN flows, across all 15 entities on the Wednesday test file, and to compare the anomaly score distributions of false positives against true negatives.

```
import pandas as pd
import sys

df = pd.read_csv(sys.argv[1])

fp = df[(df["Label"] == "BENIGN") & (df["predicted"] == -1)]
tn = df[(df["Label"] == "BENIGN") & (df["predicted"] == 1)]

print(f"Total BENIGN flows: {len(df[df['Label'] == 'BENIGN'])}")
print(f"False positives: {len(fp)}\n")

print("--- False positives by entity ---")
fp_by_entity = fp["entity"].value_counts()
total_by_entity = df[df["Label"] == "BENIGN"]["entity"].value_counts()
fpr_by_entity = (fp_by_entity / total_by_entity).sort_values(ascending=False)
print(fpr_by_entity.head(10))

print("\n--- Anomaly score distribution: FP vs TN ---")
print("False positives (wrongly flagged):")
print(fp["anomaly_score"].describe())
print("\nTrue negatives (correctly left alone):")
print(tn["anomaly_score"].describe())
```

7.4.3 Finding: The False Positive Rate Is Concentrated in Two Entities

Entity	False Positive Rate
192.168.10.50	22.4%
192.168.10.25	12.8%
Remaining 13 entities	5.2% to 7.0%, close to the calibration target

The remaining 13 entities perform close to their calibrated design target of 5 percent, meaning the 5.6 to 6 percent network wide average is being pulled upward almost entirely by these two hosts.

7.4.4 Score Distribution Sanity Check

False positives, with a mean anomaly score of negative 0.112, and true negatives, with a mean anomaly score of positive 0.122, show a large, clean separation in the underlying model scores. This means the model itself is not confused or producing noisy borderline calls near the decision boundary. It is confidently calling certain benign flows unusual. This shifts the diagnostic question from whether the model is noisy to why this specific benign traffic looks statistically different from what the model was calibrated to expect.

7.4.5 Entity 192.168.10.50: Attack-Adjacent Traffic Distortion

This is the host under active DoS attack on Wednesday. Its own non attack, benign labeled, traffic on that day differs enough from its Monday, Tuesday, and Thursday training baseline to trigger false positives, plausibly related to network conditions around or adjacent to the ongoing attack, though this was not isolated to a single specific cause beyond the broader day to day distribution drift finding established earlier in the project.

7.4.6 Entity 192.168.10.25: Reduced Activity Relative to Baseline

This is a distinct, separate cause from entity .50's. A diagnostic comparison of .25's time window features showed the following.

Metric	Baseline (Mon+Tue+Thu)	Wednesday
connections_per_minute mean	633	218, roughly 2.9 times lower
distinct_ports_per_minute mean	212	notably lower, Wednesday max only 269 versus baseline max 965

Entity .25 is quieter than its training baseline on Wednesday, not busier. Isolation Forest treats both directions of deviation, busier or quieter than learned normal, as equally anomalous, so unusually low activity gets flagged just as readily as unusually high activity, even though reduced activity is not inherently a security concern. This appears to be ordinary day to day or mid week variation in this host's usage pattern that the fixed historical baseline does not account for.

7.4.7 Conclusion

The 5.6 to 6 percent network false positive rate decomposes into two distinct, individually explainable causes rather than being generic, unexplained model noise: attack adjacent distribution drift on the directly targeted entity, and natural reduced activity variation on a second entity that the fixed baseline period does not capture. Both findings reinforce the project's earlier identified static baseline limitation. A wider, more recent, or periodically refreshed baseline window would likely absorb both directions of this variation, whereas a fixed historical snapshot cannot.

7.5 Dashboard Updates

The Streamlit dashboard required updating to reflect the new pipeline outputs from all four items above.

Data source paths were updated from the old pre time window scored and MITRE files to the new combined system outputs, `wednesday_combined.csv` and `wednesday_mitre_final.csv`.

`combine_detectors.py` was updated to output a standard predicted column alongside its `final_flag` boolean, so downstream scripts automatically treat scan detector catches identically to Isolation Forest catches with zero additional code changes.

A new MITRE technique category, T1046, Network Service Scanning, was added, triggered when the two new time window features dominate an anomaly's explanation. After regenerating the full pipeline this correctly became the single largest MITRE category, with 134,107

anomalies on the Wednesday test file, and the crosstab confirmed that DoS Hulk, a flood style attack, now correctly maps almost entirely to this scanning and burst technique rather than a generic DoS bucket, improving MITRE mapping precision as a side effect.

Risk score weighting was updated to include a severity value for the new T1046 technique, weighted at 70 out of 100, between the two existing DoS technique severities.

The alert table was updated to surface the scan_flag column, so an analyst using the dashboard can see whether a given alert was caught by the Isolation Forest model, the scan detector, or both.

A known trade off to note is that the dashboard's Flagged Anomalies count increased substantially after these changes, from approximately 211,000 to approximately 289,000 on the Wednesday file, since the combined system is deliberately more aggressive by design. This reflects the recall improvement from 68.3 to 98.1 percent, not increased noise, and is captioned accordingly rather than left to look like a regression.

7.6 Summary of Supervisor-Requested Improvements

Supervisor request	Status	Headline number
Time window features	Done	PortScan recall improved from 0.2% to 99.2% with the combined system
LOF comparison	Done	LOF beats Isolation Forest on every attack type tested; 99.4% PortScan recall standalone
Split and seed stability	Done	F1 standard deviation under 1% relative variation across five seeds for DoS and DDoS
False positive rate diagnosis	Done	Traced to 2 of 15 entities, two distinct, individually explainable causes

One item remains correctly scoped as future work rather than solved: Bot and command and control detection stays near zero across all three tested approaches, Isolation Forest, LOF, and the scan detector, consistently confirming that it requires a beaconing interval or periodicity based detector rather than any volume or density based anomaly method, which none of this work attempted to build.

8 Dashboard Screenshots

The following screenshots show the Streamlit dashboard, accessed through SSH port forwarding from the project's Ubuntu virtual machine to a local browser at <http://localhost:8501>. These screenshots reflect the dashboard state at the end of the initial remediation phase; updated screenshots reflecting the time window features, the scan detector, LOF results, and the T1046 scanning technique will be substituted once available.

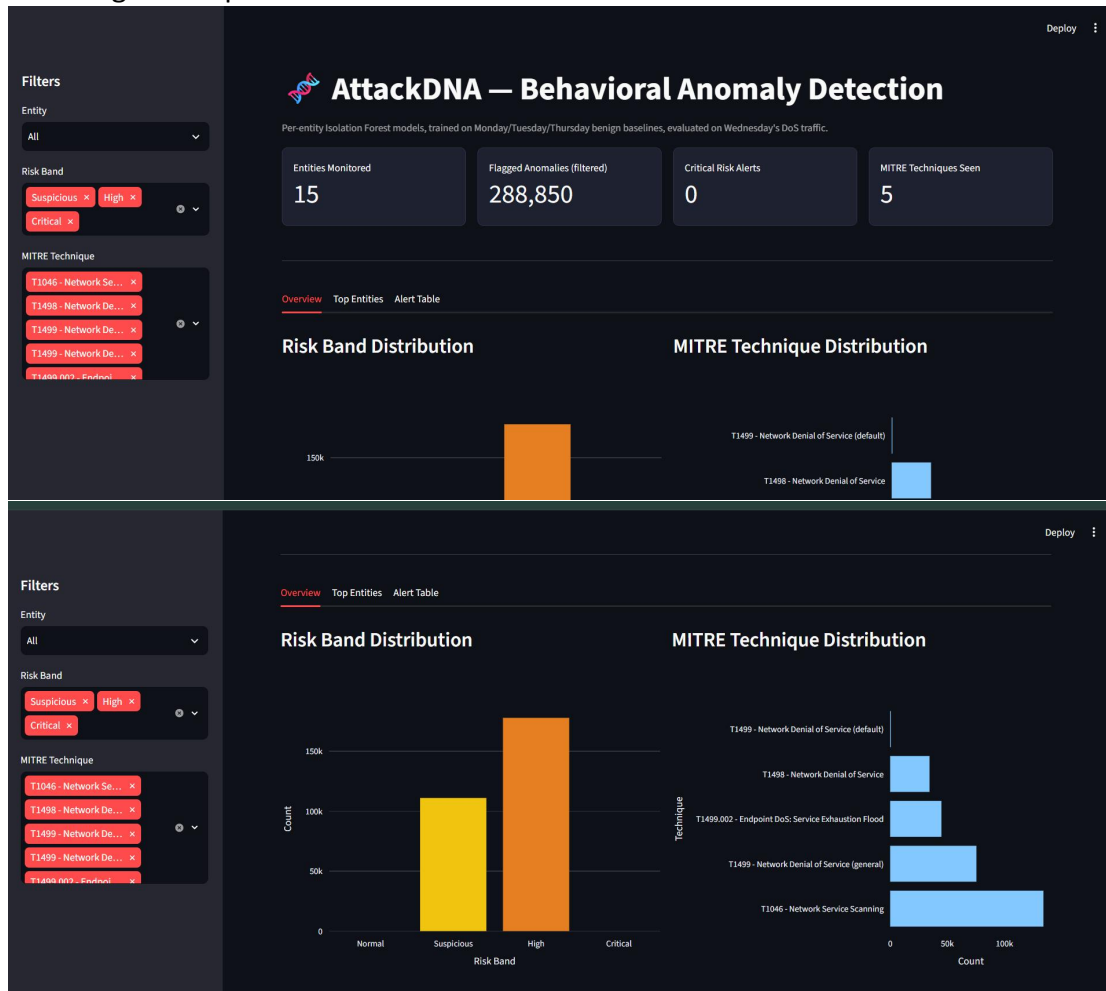


Figure 1: Overview: top level metrics and MITRE technique distribution, all 15 entities.

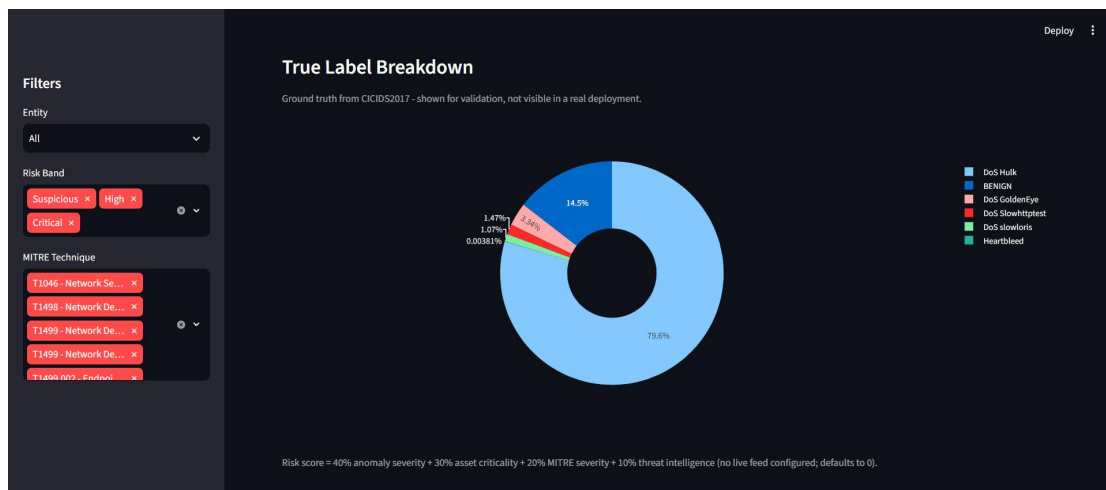


Figure 2: Risk band distribution and true label breakdown, network wide.

Entity: All

Risk Band: Suspicious, High, Critical

MITRE Technique: T1046 - Network Se..., T1498 - Network De..., T1499 - Network De..., T1499.002 - Endpoi...

Overview Top Entities **Alert Table**

Flagged Alerts (highest risk first)

	entity	risk_score	risk_band	Label	mitre_technique	mitre_reasoning
230282	192.168.10.50	79.1	High	DoS Hulk	T1498 - Network Denial of Service	High-volume flood pattern - volume evidence (83.4) outweighs timing evidence (83.4)
248692	192.168.10.50	79.1	High	DoS Hulk	T1498 - Network Denial of Service	High-volume flood pattern - volume evidence (87.2) outweighs timing evidence (87.2)
182055	192.168.10.50	79.0	High	DoS Hulk	T1498 - Network Denial of Service	High-volume flood pattern - volume evidence (83.4) outweighs timing evidence (83.4)
230276	192.168.10.50	79.0	High	DoS Hulk	T1498 - Network Denial of Service	High-volume flood pattern - volume evidence (83.4) outweighs timing evidence (83.4)
223575	192.168.10.50	79.0	High	DoS Hulk	T1498 - Network Denial of Service	High-volume flood pattern - volume evidence (86.8) outweighs timing evidence (86.8)
172138	192.168.10.50	79.0	High	DoS Hulk	T1498 - Network Denial of Service	High-volume flood pattern - volume evidence (87.2) outweighs timing evidence (87.2)
237266	192.168.10.50	79.0	High	DoS Hulk	T1498 - Network Denial of Service	High-volume flood pattern - volume evidence (87.2) outweighs timing evidence (87.2)
223820	192.168.10.50	79.0	High	DoS Hulk	T1498 - Network Denial of Service	High-volume flood pattern - volume evidence (87.2) outweighs timing evidence (87.2)
200130	192.168.10.50	79.0	High	DoS Hulk	T1498 - Network Denial of Service	High-volume flood pattern - volume evidence (87.2) outweighs timing evidence (87.2)
247922	192.168.10.50	79.0	High	DoS Hulk	T1498 - Network Denial of Service	High-volume flood pattern - volume evidence (87.2) outweighs timing evidence (87.2)
182911	192.168.10.50	79.0	High	DoS Hulk	T1498 - Network Denial of Service	High-volume flood pattern - volume evidence (87.2) outweighs timing evidence (87.2)
249091	192.168.10.50	79.0	High	DoS Hulk	T1498 - Network Denial of Service	High-volume flood pattern - volume evidence (87.2) outweighs timing evidence (87.2)
219337	192.168.10.50	79.0	High	DoS Hulk	T1498 - Network Denial of Service	High-volume flood pattern - volume evidence (87.2) outweighs timing evidence (87.2)

Figure 3: Sortable alert table showing entity, risk score, risk band, true label, MITRE technique, and plain language explanation.

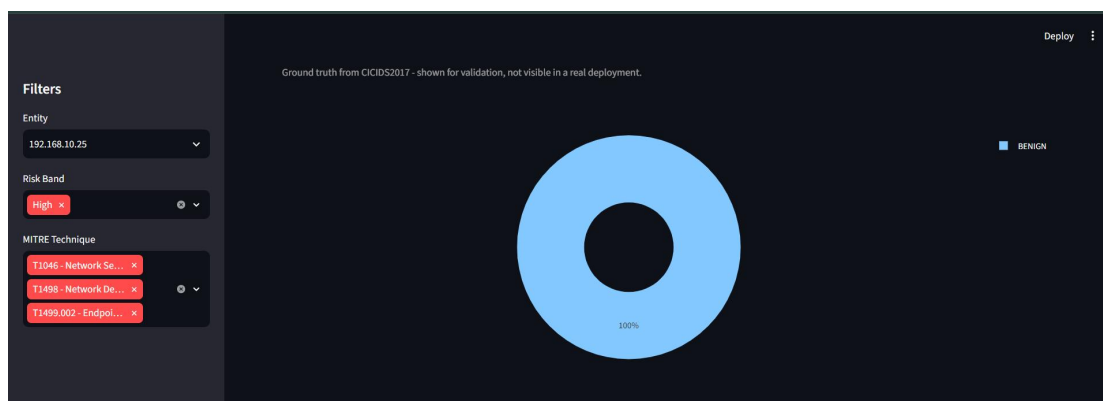


Figure 4: Entity 192.168.10.25, a zero attack entity, correctly shows zero Critical alerts after the risk scoring fix.

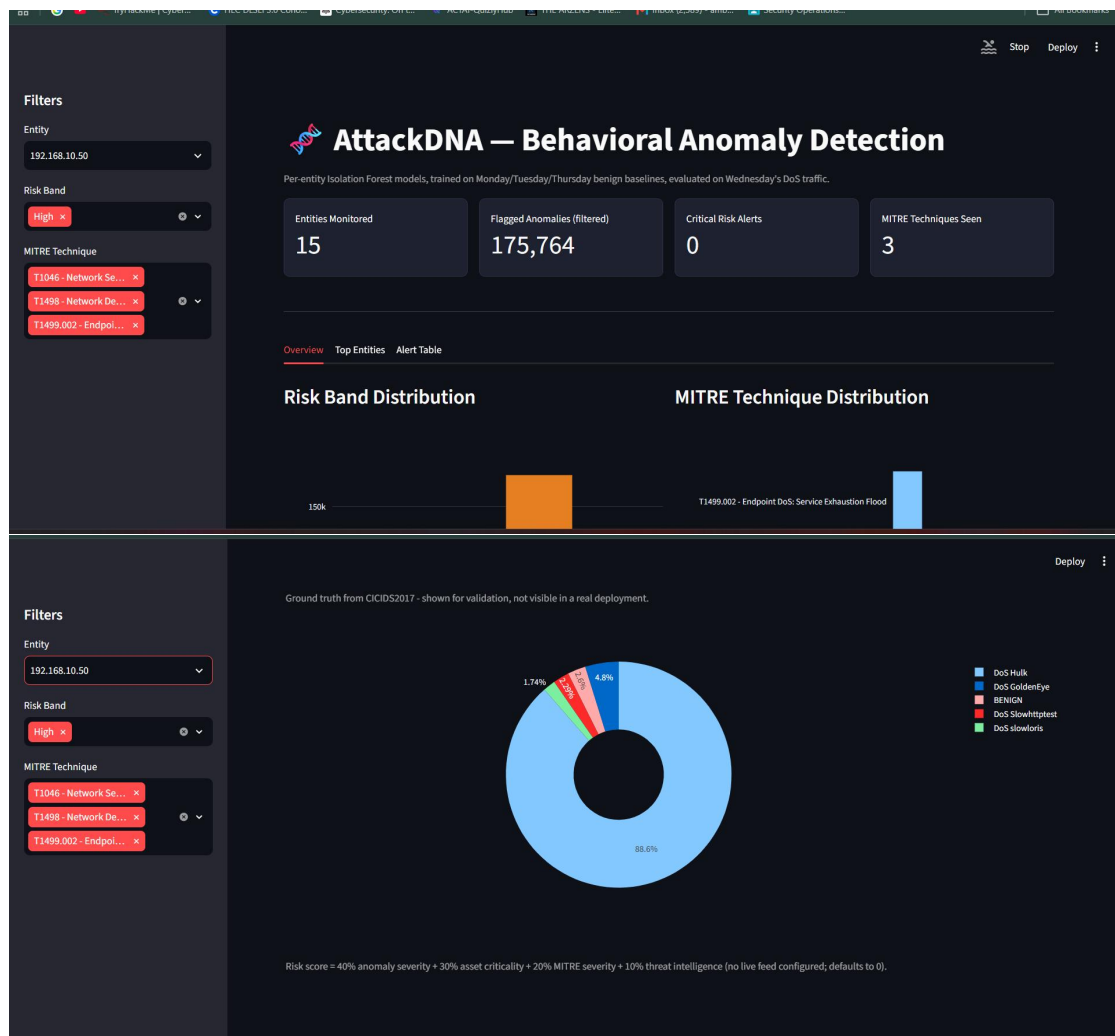


Figure 5: MITRE technique distribution for entity 192.168.10.50, the real attack target.



Figure 6: Risk band and true label breakdown for entity 192.168.10.50; 89.7 percent of its Wednesday traffic was DoS Hulk.

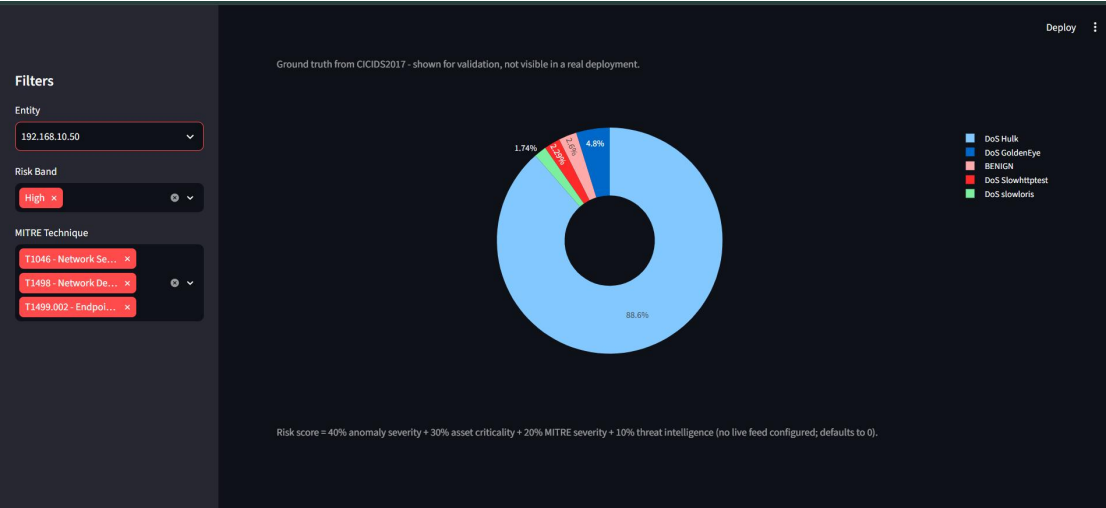
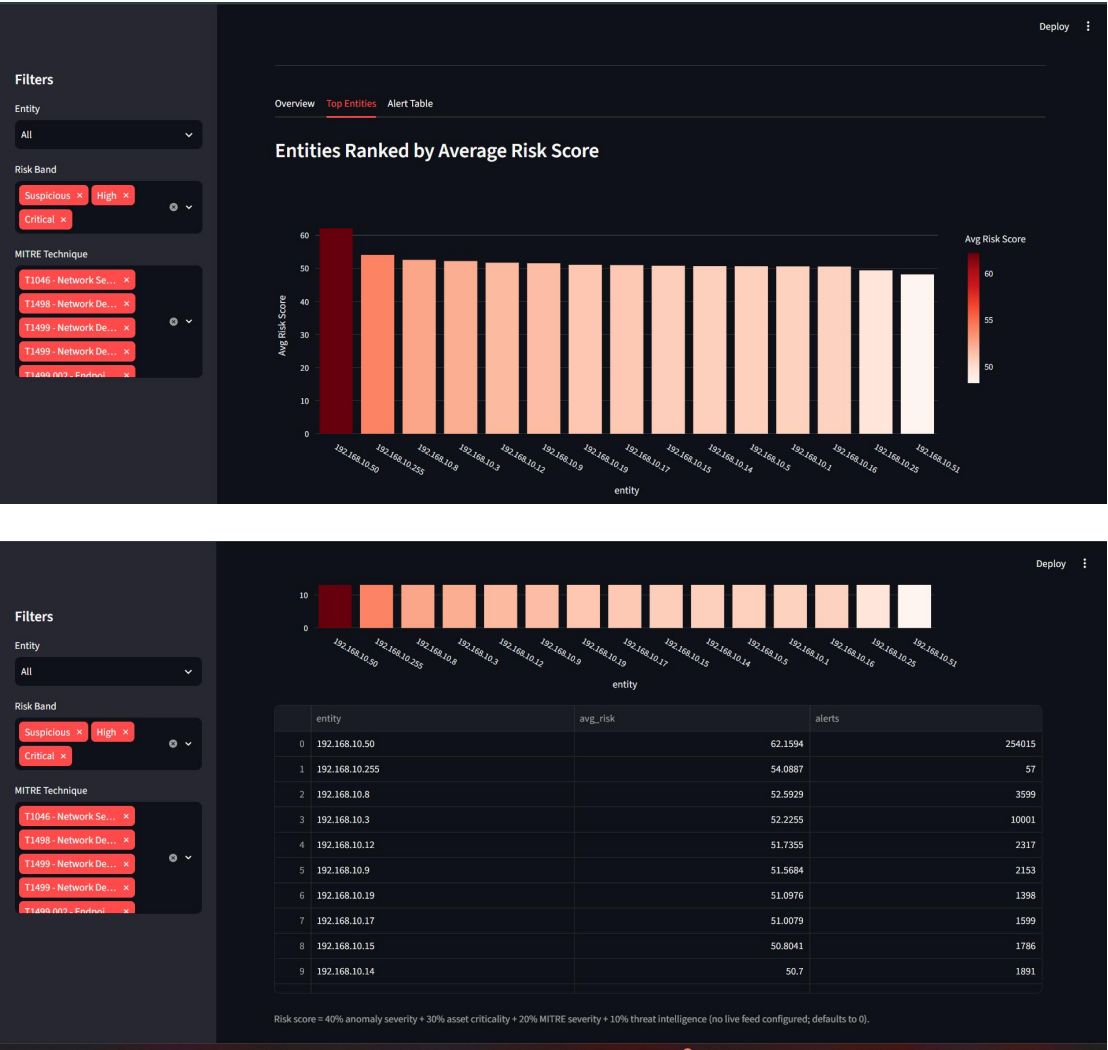
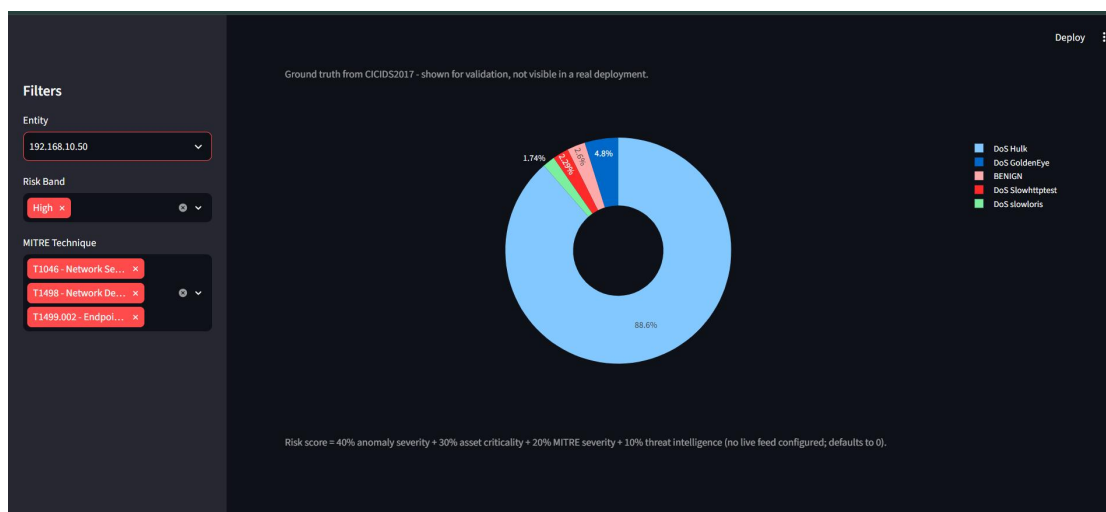


Figure 7: Top alerts for entity 192.168.10.50, correctly matched to DoS Hulk with technically accurate explanations.

Some other screenshots :





9 Discussion

An important nuance surfaced while validating the risk scoring design and is worth stating explicitly. Two designs were tried. Per entity relative scaling, where an entity's own worst anomaly always reaches close to 100, incorrectly gave a zero attack entity, 192.168.10.1, Critical alerts. Network wide global scaling, where severity is comparable across the whole network, correctly fixed the zero attack entity, but as a side effect a single high volume noisy entity, 192.168.10.3, can dominate the top of the network wide most critical ranking with false positives, while a genuinely attacked but lower volume entity may not reach the same absolute severity ranking during real attacks.

The final weighted risk score partially mitigates this by incorporating entity criticality as a separate 30 percent weighted term, so a manually flagged critical asset such as the web server is boosted regardless of its raw anomaly score ranking. A more complete solution, combining each flow's global percentile with its entity local percentile, for example by averaging the two or taking the maximum, is identified as a natural next refinement.

The supervisor requested improvements also surfaced a second, equally important nuance: adding a strong new feature is not sufficient on its own if the underlying algorithm has a structural blind spot for the pattern that feature reveals. The time window features contained an extremely strong statistical signal, roughly 90 standard deviations of separation for the targeted entity during the PortScan attack, and yet Isolation Forest still failed to use it, because the attack traffic formed a dense, self similar cluster rather than a handful of scattered outliers. This is a genuinely useful lesson for the report: feature engineering and algorithm choice are not independent decisions, and a mismatch between the two can hide a signal that is otherwise obviously present in the data.

10 Conclusion and Future Work

10.1 Summary

AttackDNA demonstrates a complete, working, per entity behavioral anomaly detection pipeline: data cleaning, per entity model training on genuinely trained, not API based, models, calibrated detection, statistical explanation generation, rule based MITRE ATT&CK mapping, and an interactive dashboard, all built with free, open source, CPU only tooling. Following supervisor review, the system was extended with time window burst features and a dedicated scan detector, closing the PortScan detection gap from 0.2 percent to 99.2 percent recall, benchmarked against a second algorithm, Local Outlier Factor, which outperformed Isolation Forest on every attack type tested, validated for result stability across five random seeds, and had its baseline false positive rate traced to two specific, individually explainable causes rather than left as unexplained noise.

10.2 Recommended Priority for Future Work

Beaconing or periodicity based detection for Bot and command and control traffic, the one weakness that persisted across every approach tested in this project.

Testing LOF combined with the scan detector layer, which was identified but not yet run.

A sliding window implementation for the time window features, to remove the fixed bucket boundary edge effect documented in Section 7.1.

Model native explanations, using SHAP or LIME, for why the model decided, rather than only what changed statistically.

Real asset criticality values sourced from an actual asset inventory, replacing the current placeholder defaults.

Live detection using streaming infrastructure, a genuine follow on infrastructure project, out of scope for the current CPU only, no live services constraint.

10.3 Closing Note

This project intentionally reports negative and mixed results alongside positive ones, including the baseline expansion that barely helped, the Hulk MITRE mapping that splits roughly evenly between categories, and the Bot detection gap that persisted across three different technical approaches, because an evaluation that only shows success is less credible, and less useful, than one that honestly identifies where an approach works well and where it does not. The strongest single output of this work is arguably not a metric, but the diagnosis in Section 7.1: proving, with controlled evidence, why a specific class of attack was structurally invisible to the original detection design, and then engineering a targeted, well justified fix for it.

Project Repository: <https://github.com/Amber8/attackdna>